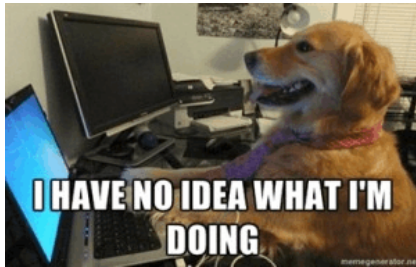


Por fin tenemos reference.



Contents

1 Descripción de Algoritmos	2
1.1 Data structures	2
1.2 Graphs	4
1.3 Math	5
1.4 Geometry	6
1.5 Strings	7
1.6 Other	7
2 Data structures	8
2.1 Segment tree	8
2.2 Segment tree - Lazy propagation	8
2.3 Segment tree - Persistence	9
2.4 Segment tree - 2D	9
2.5 Sparse table (static RMQ)	10
2.6 Fenwick tree	10
2.7 Wavelet tree	10
2.8 STL extended set	11
2.9 STL rope	11
2.10 Treap (as BST)	11
2.11 Treap (implicit key)	12
2.12 Treap (with node father)	12
2.13 Link-Cut tree	13
2.14 Convex hull trick (static)	14
2.15 Convex hull trick (dynamic)	14
2.16 Li-Chao tree	14
2.17 Gain-cost-set	15
2.18 Disjoint intervals	15
3 Graphs	16
3.1 Topological sort	16

3.2 Kruskal (+ Union-Find)	16
3.3 Dijkstra	16
3.4 Bellman-Ford	16
3.5 Floyd-Warshall	17
3.6 Strongly connected components (+ 2-SAT)	17
3.7 Articulation - Bridges - Biconnected	17
3.8 Chu-Liu (minimum spanning arborescence)	18
3.9 LCA - Binary Lifting	18
3.10 Heavy-Light decomposition	19
3.11 Centroid decomposition	19
3.12 Parallel DFS	19
3.13 Eulerian path	20
3.14 Dynamic connectivity	20
3.15 Dominator tree	21
3.16 Edmond's blossom (matching in general graphs)	21
4 Math	22
4.1 Identities	22
4.2 Theorems	22
4.3 Integer floor division	23
4.4 Sieve of Eratosthenes	23
4.5 Generate divisors	23
4.6 Pollard's rho	23
4.7 Simpson's rule	24
4.8 Polynomials	24
4.9 Bairstow	25
4.10 Fast Fourier Transform	26
4.11 FFT operations	27
4.12 Fast Hadamard Transform	29
4.13 Karatsuba	29
4.14 Diophantine	30
4.15 Modular inverse	30
4.16 Chinese remainder theorem	30
4.17 Mobius	30
4.18 Matrix exponentiation	30
4.19 Matrix reduce and determinant	31
4.20 Simplex	31
4.21 Discrete log	32
4.22 Berlekamp Massey	32
4.23 Linear Rec	32
4.24 Tonelli Shanks	33
4.25 Prime count	33

5	Geometry	33
5.1	Point	33
5.2	Line	34
5.3	Circle	34
5.4	Polygon	35
5.5	Plane	37
5.6	Radial order of points	37
5.7	Convex hull	37
5.8	Dual from planar graph	37
5.9	Halfplane intersection	38
5.10	KD tree	38
5.11	Minkowski sum	39
5.12	Delaunay triangulation	39
6	Strings	41
6.1	KMP	41
6.2	Z function	41
6.3	Manacher	41
6.4	Aho-Corasick	41
6.5	Suffix automaton	42
6.6	Palindromic Tree	42
6.7	Suffix array (shorter but slower)	42
6.8	Suffix array	43
6.9	LCP (Longest Common Prefix)	43
6.10	Suffix Tree (Ukkonen's algorithm)	43
6.11	Hashing	44
6.12	Hashing with ll (using __int128)	44
7	Flow	44
7.1	Matching (slower)	44
7.2	Matching (Hopcroft-Karp)	45
7.3	Hungarian	45
7.4	Dinic	45
7.5	Min cost max flow	46
8	Other	47
8.1	Mo's algorithm	47
8.2	Divide and conquer DP optimization	47
8.3	Dates	47
8.4	C++ stuff	48
8.5	Interactive problem tester template	48
8.6	Max number of divisors up to 10^n	48
8.7	Template	48

1 Descripción de Algoritmos

1.1 Data structures

Segment tree — Complexity: $O(N)$ de construcción, $\log N$ por operación)

Usage: 1. `STree(int n)` → Reserva el espacio para un arreglo de tamaño n . 2.

`upd(int p, int v)` → Reemplaza el elemento en la posición p por el valor v . 3. `query(int l, int r)` → Devuelve el resultado (ej. mínimo o suma) de los elementos en el rango $[l, r)$.

Note: Útil para hacer consultas rápidas sobre un segmento de un arreglo cuando los elementos individuales cambian constantemente. Usa indexación base 0. Para cambiar la operación (ej. de mínimo a máximo), adapta el elemento neutro y la función de combinación.

Segment tree lazy — Complexity:

$O(N)$ de construcción, $\log N$ por actualización/consulta)

Usage: 1. `STree(int n)` → Crea el árbol preparado para "lazy propagation". 2.

`upd(int l, int r, int v)` → Aplica la modificación v (ej. sumar v) a todos los elementos dentro del rango $[l, r)$ al mismo tiempo. 3. `query(int l, int r)` → Devuelve el resultado del rango $[l, r)$.

Note: Igual al Segment Tree, pero este te permite modificar un bloque completo de datos en un solo paso (ej. sumar 5 a todos del índice 2 al 10), sin actualizar celda por celda de manera ineficiente.

Persistent segment tree — Complexity:

$O(\log N)$ por versión de actualización/consulta)

Usage: 1. `STree(int n)` → Inicializa el árbol. 2. `upd(int root, int p, int v)` → Toma la "foto" del árbol en la versión `root`, cambia la posición p a v , y devuelve el identificador de la nueva versión creada. 3. `query(int root, int l, int r)` →

Consulta el rango $[l, r)$ viajando en el tiempo hacia la versión específica `root`.

Note: Es una máquina del tiempo para arreglos. Cada actualización no sobrescribe el pasado, sino que crea una línea temporal alterna. Debes guardar externamente los identificadores devueltos por `upd` para consultar el pasado.

2D segment tree — Complexity:

$O(NM)$ de construcción, $\log N \log M$ por operación)

Usage: 1. `build()` → Inicializa el árbol leyendo una matriz global predefinida. 2.

`upd(int x, int y, int v)` → Cambia el valor en la coordenada (x, y) por v . 3. `query(int x0, int x1, int y0, int y1)` → Devuelve el resultado dentro del rectángulo delimitado por las filas $[x0, x1)$ y columnas $[y0, y1)$.

Note: Extensión directa para trabajar en grillas 2D. Asegúrate de configurar las variables globales N y M (dimensiones de la matriz) antes de usarlo.

Sparse table — Complexity: $O(N \log N)$ de preálculo, $O(1)$ por consulta)

Usage: 1. `st_init(int *a)` → Procesa el arreglo inicial `a` para preparar las respuestas. 2. `st_query(int s, int e)` → Devuelve la respuesta instantánea en el

rango $[s, e)$.

Note: Estructura estática. Solo úsala si el arreglo **nunca** cambia y necesitas responder consultas instantáneamente. Exclusivo para operaciones donde repetir elementos no altera el resultado (idempotentes: min, max, gcd). No sirve para sumas.

Fenwick tree — Complexity: $O(\log N)$ por actualización/consulta)

Usage: 1. `upd(int i0, int v)` → Le SUMA el valor v a lo que ya exista en la posición $i0$. 2. `get(int i0)` → Devuelve la suma acumulada desde el inicio hasta la posición antes de $i0$. 3. `get_sum(int i0, int i1)` → Suma del rango $[i0, i1)$.

Note: Es la versión ligera y rápida de un Segment Tree. Excelente cuando solo necesitas sumas de prefijos y actualizaciones de un solo elemento.

Wavelet tree — Complexity: $O(N \log \sigma)$ de construcción, $\log \sigma$ por consulta)

Usage: 1. `WT(int n)` → Inicializa para n elementos. 2. `add(int v)` → Agrega un valor al final. 3. `range(int k, int s, int e, int a, int b)` → Cuenta cuántos números caen en el rango de valores $[a, b)$ buscando únicamente dentro del subarreglo desde s hasta e .

Note: Como un segment tree pero separa valores por bits. Ideal para consultas tipo "cuántos números menores a X hay en este rango del arreglo". Debes comprimir valores muy grandes antes de insertarlos.

STL extended set — Complexity: $O(\log N)$ por operación)

Usage: 1. `ordered_set` → Úsalo como un set de C++ normal. 2. `find_by_order(k)` → Devuelve un iterador al k -ésimo elemento más pequeño (0-indexado). 3. `order_of_key(x)` → Cuenta exactamente cuántos elementos en el set son estrictamente menores que x .

Note: Es una caja negra maravillosa que le añade acceso por índice a un Set normal. Requiere directivas GNU de C++, por lo que solo compila en GCC (como en casi todos los jueces).

STL rope — Complexity: $O(\log N)$ por separación/inserción/borrado)

Usage: 1. `rope<T>` → Declara tu estructura como si fuera un string o vector. 2. `insert(pos, value)` → Inserta `value` en la posición `pos` de manera eficiente. 3. `erase(pos, len)` → Borra `len` elementos a partir de `pos`.

Note: Alternativa para no programar un Treap. Útil si necesitas cortar, pegar y modificar grandes pedazos de un arreglo o cadena de texto. Requiere GNU C++.

Treap as BST — Complexity: $O(\log N)$ esperado por operación)

Usage: 1. `split(pitem t, int key, pitem& l, pitem& r)` → Parte el árbol en dos: los menores a `key` en `l`, y el resto en `r`. 2. `insert(pitem& t, pitem it)` → Inserta el nodo `it`. 3. `kth(pitem t, int k)` → Encuentra el k -ésimo menor en el árbol.

Note: Es un árbol de búsqueda binaria (como un map/set) balanceado mediante azar. Debes asignar una prioridad aleatoria a cada nodo al crearlo.

Implicit treap — Complexity: $O(\log N)$ esperado por operación)

Usage: 1. `merge(pitem& t, pitem l, pitem r)` → Pega el arreglo `r` justo a la derecha del arreglo `l`. 2. `split(pitem t, pitem& l, pitem& r, int sz)` → Corta el arreglo dejando los primeros `sz` elementos en `l` y el resto en `r`. 3. `output(pitem t)` → Imprime o recupera el arreglo resultante en orden.

Note: Funciona como un arreglo mágico en el que puedes cortar un pedazo del medio y pegarlo al inicio en tiempo logarítmico. Las claves son implícitas según la posición.

Implicit treap with father — Complexity: $O(\log N)$ esperado por operación)

Usage: Igual al Implicit treap normal, pero incluye punteros al padre. 1. `root(pitem t, int& pos)` → A partir de un nodo aislado, escala hacia arriba para encontrar la raíz y determina qué posición exacta ocupa este nodo dentro del arreglo completo.

Note: Útil cuando el problema te da una referencia directa a un elemento y te pregunta "¿en qué posición está actualmente este elemento después de tantas mezclas?".

Link-cut tree — Complexity: $O(\log N)$ amortizado por operación)

Usage: 1. `evert(pitem v)` → Gira el árbol para que `v` sea la nueva raíz suprema. 2. `link(pitem v, pitem w)` → Dibuja una nueva arista conectando `v` con `w`. 3. `cut(pitem v, pitem w)` → Destruye la arista entre `v` y `w`.

Note: Mantiene un bosque de árboles que cambia dinámicamente. Útil cuando te piden añadir o borrar aristas sobre la marcha y responder preguntas de conectividad.

Link-cut tree with path aggregate — Complexity:

$O(\log N)$ amortizado por operación)

Usage: 1. `mkR(Node x)` → Hace a `x` raíz. 2. `link(Node x, Node y)` → Conecta nodos. 3. `query(Node x, Node y)` → Te da la respuesta combinada (ej. suma, máximo) de todos los nodos en el camino más corto entre `x` e `y`.

Note: Una versión del Link-Cut Tree equipada para responder sobre caminos. Configura las funciones agregadas según lo que necesites calcular.

Static convex hull trick — Complexity:

$O(\log N)$ por consulta, tras inserción monótona)

Usage: 1. `CHT::add(Line l)` → Agrega una nueva ecuación de recta $y = mx + b$. Ojo: las pendientes m deben insertarse estrictamente ordenadas. 2. `CHT::query(tc x)` → Dado un valor de x , evalúa en todas las rectas insertadas y te devuelve el valor de y más pequeño.

Note: Ideal para optimizar Programación Dinámica. Si buscas máximos en vez de mínimos, multiplica tus pendientes y constantes por -1 , y multiplica el resultado por -1 .

Dynamic convex hull trick — Complexity: $O(\log N)$ por inserción/consulta)

Usage: 1. `HullDynamic::add_line(tc m, tc b)` → Agrega una recta en cualquier orden, sin restricciones. 2. `HullDynamic::query(tc x)` → Devuelve el valor **máximo** al evaluar x .

Note: Funciona igual que el estático pero no requiere que las pendientes lleguen ordenadas. Por defecto, esta implementación busca el MÁXIMO. Para buscar

mínimos, inserta $(-m, -b)$ y niega la respuesta.

Li Chao tree — Complexity: $O(\log C)$ por inserción/consulta)

Usage: 1. `LiChaoTree(T xmin, T xmax)` → Define el rango válido de valores x que podrías consultar. 2. `add_line(Line nw)` → Inserta una recta o segmento de recta.

3. `query(T x)` → Encuentra el menor valor de y posible en el punto x evaluando todas las funciones insertadas.

Note: Es una alternativa moderna y más fácil de modificar que el Convex Hull Trick dinámico.

Gain-cost set — Complexity: $O(\log N)$ amortizado por inserción/consulta)

Usage: 1. `GCS::add(ii x)` → Agrega una opción descrita por un par y automáticamente descarta las opciones dominadas (peores en todo). 2.

`GCS::query(int v)` → Calcula el mejor rendimiento dado tu recurso v .

Note: Mantiene una "Frontera de Pareto" limpia usando un set de C++.

Disjoint intervals — Complexity: $O(K \log N)$ por inserción)

Usage: 1. `disjoint_intervals::add(ii v)` → Agrega un nuevo intervalo de tiempo o espacio al sistema. Si choca o se empalma con intervalos ya existentes, los fusiona automáticamente.

Note: Ideal para colapsar un montón de rangos superpuestos y saber exactamente cuáles son las islas de cobertura resultantes.

1.2 Graphs

Topological sort — Complexity: $O(V + E)$

Usage: 1. `tsort()` → Ordena todos los nodos del grafo dirigido de tal forma que ninguna arista apunte hacia atrás. Si hay empates, devuelve el orden lexicográficamente más pequeño.

Note: Resuelve problemas de dependencia ("hacer A antes que B"). Si el arreglo resultante tiene menos nodos que V , significa que hay un ciclo (paradoja).

Kruskal MST — Complexity: $O(E \log E)$

Usage: 1. `uf_init()` → Prepara los nodos para Union-Find. 2. Ordena tu lista global de aristas de menor a mayor peso. 3. `kruskal()` → Recorre las aristas y devuelve el costo total de conectar todos los nodos sin formar ciclos.

Note: Árbol de Expansión Mínima. Úsalo para tender redes eléctricas de mínimo costo que conecten todas las ciudades.

Dijkstra — Complexity: $O((V + E) \log V)$

Usage: 1. `dijkstra(int x)` → Explora el mapa buscando la ruta más corta desde x hacia el resto de nodos. El arreglo `dist[]` guardará los costos mínimos.

Note: El clásico algoritmo de rutas. Exige que ninguna calle tenga peso negativo. Nodos inaccesibles quedarán marcados con infinito (o -1 según la implementación).

Bellman-Ford — Complexity: $O(VE)$

Usage: 1. `bford(int src)` → Calcula distancias desde `src`. Propaga adecuadamente las reducciones por ciclos negativos.

Note: Es como Dijkstra pero soporta costos negativos. Útil para problemas con máquinas del tiempo o intercambios con ganancia neta. Detecta ciclos infinitos de recolección.

Floyd-Warshall — Complexity: $O(V^3)$

Usage: 1. Inicializa tu matriz `g[u][v]` con el peso directo de cada arista y cero en la diagonal. 2. `floyd()` → Transforma la matriz en las distancias más cortas absolutas entre TODO par de nodos.

Note: Rápido de programar pero muy lento en tiempo. Úsalo solo para grafos pequeños ($V \leq 400$).

SCC and 2-SAT — Complexity: $O(V + E)$

Usage: 1. `scc()` → Agrupa los nodos en Componentes Fuertemente Conexas. 2. `addor(int a, int b)` → Agrega una restricción tipo "o A o B deben cumplirse". 3.

`satisf(int nvar)` → Devuelve verdadero si existe una configuración lógica que cumpla todas las reglas.

Note: La herramienta para problemas de decisiones binarias encadenadas, como rompecabezas de restricciones donde no puedes tener contradicciones lógicas.

Articulation, bridges and biconnected — Complexity: $O(V + E)$

Usage: 1. `doit()` → Analiza el grafo. Al terminar, la bandera `edge.bridge` indica si destruir esa arista específica partiría la red de comunicación en pedazos aislados.

Note: Sirve para hallar los puntos débiles de una estructura o una red de servidores.

Chu-Liu Edmonds arborescence — Complexity: $O(VE)$

Usage: 1. `ChuLiu(int n, int r)` → Prepara instancia. 2. `doit()` → Devuelve el costo de la red mínima dirigida desde la raíz r .

Note: Es el análogo de Kruskal (MST) pero para calles de UN SOLO sentido.

Garantiza rutas válidas dirigidas desde la base r hacia todo el mundo.

LCA binary lifting — Complexity: $O(N \log N)$ preálculo, $\log N$ por consulta)

Usage: 1. `lca_init()` → Recorre el árbol y anota saltos de tamaño potencia de 2. 2. `lca(int x, int y)` → Devuelve el nodo Ancestral Común más profundo entre x e y .

Note: Vital para resolver distancias reales en árboles usando la fórmula:

$$\text{dist}(u, v) = \text{prof}(u) + \text{prof}(v) - 2 \cdot \text{prof}(\text{LCA}(u, v)).$$

Heavy-light decomposition — Complexity:

$O(N)$ inicialización, $\log^2 N$ por consulta)

Usage: 1. `hld_init()` → Acomoda el árbol en un arreglo recto. 2. `query(int x, int y, STree& rmq)` → Consulta el segmento continuo de las ramas del árbol apoyándose en un Segment Tree.

Note: Herramienta definitiva para aplicar modificaciones o cálculos sobre trayectorias dentro de un árbol como si fueran segmentos de un arreglo 1D.

Centroid decomposition — Complexity: $O(N \log N)$

Usage: 1. `centroid()` → Encuentra el "centro de gravedad" del árbol y lo extrae recursivamente. 2. Dentro de `cdfs`, programas lo que deseas calcular con las mitades particionadas.

Note: Divide el problema de conteo de caminos en mitades. Usado para problemas tipo "¿cuántos caminos en el árbol suman K de longitud?".

Parallel DFS — Complexity: $O(V + E)$

Usage: 1. `Tree::go(int x)` → Ejecuta una exploración simultánea.

Note: Permite recorrer y comparar varios árboles/grafos a la vez manejando su estado conjunto.

Eulerian path — Complexity: $O(V + E)$

Usage: 1. `add_edge(int a, int b)` → Agrega los trazos. 2. `get_path(int x)` → Devuelve un arreglo con el orden de vértices a pisar para recorrer TODAS las aristas exactamente una vez.

Note: El problema de los "Puentes de Königsberg" o "dibujar sin levantar el lápiz". Revisa la factibilidad de los grados (par/impar) de los nodos antes de llamarla.

Offline dynamic connectivity — Complexity: $O((N + Q) \log Q)$

Usage: 1. `add(int l, int r, Query e)` → Especifica entre qué turnos $[l, r]$ existe cierta arista. 2. `go(int k, int s, int e)` → Ejecuta el historial temporal usando rollback sobre un DSU.

Note: Caja negra para procesar queries de grafos que sufren alteraciones y borrados. Requiere guardar todas las consultas de antemano ("offline").

Dominator tree — Complexity: $O((V + E)\alpha(V))$

Usage: 1. `dominators(int rt)` → Rellena un arreglo `idom[v]` que indica qué vértice actúa como aduana obligatoria y directa para llegar a v desde rt .

Note: Halla nodos críticos inescapables en grafos dirigidos.

Edmonds blossom — Complexity: $O(V^2E)$

Usage: 1. `edmonds()` → Devuelve el tamaño máximo de emparejamientos sin superposición que se pueden formar en el grafo.

Note: Emparejamiento máximo para "Grafos Generales". Úsalo cuando las reglas te impiden dividir los nodos en dos grupos (bipartito) separados.

1.3 Math

Integer floor division — Complexity: $O(1)$

Usage: 1. `floordiv(ll x, ll y, ll& q, ll& r)` → Calcula la división hacia $-\infty$ y deposita el cociente en q y el residuo positivo/concordante en r .

Note: C++ estándar trunca hacia 0, lo que genera errores garrafales con coordenadas negativas. Usa esto para que las matemáticas modulares cuadren perfecto sin importar el signo.

Extended Euclid — Complexity: $O(\log \min(a, b))$

Usage: 1. `euclid(ll a, ll b, ll& x, ll& y)` → Calcula el Máximo Común Divisor (GCD) entre a y b , y averigua los factores x, y para resolver $ax + by = \text{GCD}$.

Note: Imprescindible en la búsqueda del Inverso Modular y Teorema Chino del Resto.

Sieve of Eratosthenes — Complexity: $O(N \log \log N)$

Usage: 1. `init_sieve()` → Elimina múltiplos velozmente dejando registrados únicamente los números primos hasta un límite máximo (MAXN).

Note: Llama esta función una sola vez al inicio. Ideal cuando tienes múltiples casos de prueba que requieren factorizar o preguntar la primalidad de números pequeños-medianos ($< 10^7$).

Generate divisors — Complexity: $O(d(n))$

Usage: 1. `divisors(vector<pair<ll,int>> f)` → A partir de la huella prima (ej. $2^2 \times 3^1$), genera la lista completa de todos los divisores enteros posibles.

Note: Útil y rápida cuando necesitas saber entre cuántas partes exactas se puede dividir un número ya factorizado.

Pollard Rho factorization — Complexity: $O(N^{1/4})$

Usage: 1. `rabin(ll n)` → Test de primalidad para números ridículamente grandes (10^{18}). 2. `fact(ll n, map<ll,int>& f)` → Desmenuza un número gigante guardando su factorización en el mapa f .

Note: La salvación absoluta cuando la criba de Eratóstenes explota por falta de RAM. Usa aleatoriedad para hallar los divisores más escurridizos al instante.

Simpson integration — Complexity: $O(N)$

Usage: 1. `integrate(double f(double), double a, double b, int n)` → Aproxima el área bajo la curva de la función matemática f usando n subintervalos.

Note: Usa n par y grande (100000) para excelente precisión en problemas de áreas continuas y curvas de cálculo.

Polynomial utilities — Complexity: $O(\text{Depende})$

Usage: 1. `poly<T>` → Úsalo como tipo de dato de tus ecuaciones. 2. `roots(poly<> p)` → Halla raíces enteras. 3. `interpolate(x, y)` → Halla la ecuación que cruza dichos puntos.

Note: Permite manipular polinomios sin complicarte con índices manuales.

Bairstow roots — Complexity: $O(\text{ITER} \cdot \text{Grado})$

Usage: 1. `bairstow(poly<> p)` → Retira y calcula aproximaciones cuadráticas del polinomio mayor.

Note: Método numérico iterativo potente para descifrar raíces complejas o reales reduciendo el polinomio.

Fast Hadamard Transform — Complexity: $O(N \log N)$

Usage: 1. `fht(ll* p, int n, bool inv)` → Transforma el arreglo al dominio de Walsh. 2. `multiply(vector<ll>& p1, vector<ll>& p2)` → Convoluciona los elementos operando bajo reglas bit a bit (XOR, AND, OR).

Note: Sirve para responder rápidamente combinaciones de bits (ej. "de cuántas formas saco el número C sumando un ítem A y un ítem B usando la operación XOR"). Modifica el símbolo interno según la operación deseada.

Karatsuba multiplication — Complexity: $O(N^{1.585})$

Usage: 1. `multiply(vector<tp> p0, vector<tp> p1)` → Multiplica (convoluciona) dos polinomios/números masivos rompiéndolos recursivamente.

Note: La alternativa al FFT. Úsalo si tu entorno tiene problemas con módulos y números flotantes que arruinan la precisión del FFT tradicional.

Linear Diophantine equations — Complexity: $O(\log \min(a, b))$

Usage: 1. `diophantine(ll a, ll b, ll r)` → Encuentra los valores X e Y enteros que satisfacen $A \cdot X + B \cdot Y = R$.

Note: La condición de oro es que R sea obligatoriamente un múltiplo divisible por el $\text{GCD}(A, B)$, en caso contrario no existirá solución entera.

Modular inverse — Complexity: $O(\log \text{mod})$

Usage: 1. `inv(ll a, ll mod)` → Encuentra el número por el que debes multiplicar para simular una división entre a bajo la aritmética de un módulo de reloj.

Note: a y el `mod` deben ser coprimos ($\text{GCD} = 1$) para que el inverso exista.

Simplex Algorithm — Complexity:

Exponencial en teoría, muy rápido en práctica

Usage: 1. Traduce tu problema a ecuaciones matemáticas, cárgalas en la matriz de la función y recibe el máximo (o mínimo) valor posible de tu función objetivo.

Note: La bala de plata para "Tengo X recursos y condiciones variadas; quiero maximizar mi ganancia monetaria sin violar los límites". (Programación Lineal).

Discrete logarithm — Complexity: $O(\sqrt{M})$

Usage: 1. `discrete_log(ll a, ll b, ll m)` → Resuelve el logaritmo en un reloj: encuentra cuántas veces se debe potenciar a para que resulte ser b bajo módulo m .

Note: Utiliza el ingenioso algoritmo Baby-Step Giant-Step para acortar el tiempo de M a su raíz cuadrada.

Berlekamp-Massey — Complexity: $O(N^2)$

Usage: 1. `BM(vi x)` → Dale un puñado de números de una secuencia repetitiva desconocida, y este algoritmo deducirá la relación/fórmula lineal que la produce y sus coeficientes exactos.

Note: Adulteración de DP puro. Si no logras pensar la lógica del problema, usa backtracking bruto, escupe los primeros términos de respuesta, mételes en BM y te dará el DP de regalo.

Linear recurrence — Complexity: $O(K^2 \log N)$

Usage: 1. `LinearRec` y `calc(ll k)` → Dado tu estado inicial y los coeficientes de recurrencia (obtenidos a mano o por BM), halla instantáneamente el término súper-lejos (ej. 10^{18}) de la sucesión.

Note: Trabaja bajo la misma teoría mágica que Exponenciación de Matrices pero de

forma polinómica.

Tonelli-Shanks — Complexity: $O(\log^2 P)$

Usage: 1. `tonelli_shanks(ll n, ll p)` → Dado un número n y un módulo primo p , extrae el valor X que cumple $X^2 \equiv n \pmod{p}$.

Note: Tu calculadora para Raíz Cuadrada en aritmética modular. Si la función `legendre` devuelve -1, significa que no existe ninguna raíz bajo las reglas de ese módulo.

Prime counting — Complexity: $O(N^{2/3} / \log^{1/3} N)$

Usage: 1. `count_primes(ll n)` → Devuelve exactamente el número de primos menores o iguales a n .

Note: Espectacular algoritmo para cuando n es exagerado (10^{11}) y Eratóstenes falla, pero el problema es simple: "dime la cantidad de primos".

1.4 Geometry

Point / Vector operations — Complexity: $O(1)$

Usage: 1. `cross(p1, p2)` → Producto Cruz; calcula áreas y dictamina si el polígono gira a izquierda o derecha. 2. `dot(p1, p2)` → Producto Punto; calcula proyecciones.

Note: La base de la vida geométrica. El producto cruz salva la vida al esquivar funciones de ángulos con coma flotante, evitando errores catastróficos de aproximación.

Line / Segment operations — Complexity: $O(1)$

Usage: 1. `operator^(ln l, ln m)` → Ejecuta y extrae exactamente el punto de intersección entre las dos rectas.

Note: Mantén un margen EPS sano para no comparar 0.0 exacto por temas de decimales flotantes.

Circle — Complexity: $O(1 \text{ o } N \text{ para interacciones})$

Usage: 1. `circle(pt o, double r)` → Crea y clasifica círculos. 2.

`intercircles(vector<circle> c)` → Estima el área consolidada de múltiples círculos superpuestos como en un diagrama de Venn masivo.

Note: Altamente sensible al parámetro EPS.

Polygon — Complexity: $O(N)$

Usage: 1. `pol::area()` → Calcula el área orientada internamente dividiendo todo en triángulos desde el origen mediante el producto cruz. 2. `pol::has(pt q)` →

Confirma si el punto q se encuentra dentro de los dominios del polígono.

Note: Revisa que la lista de vértices esté ordenada y respete la convexidad y dirección que espera tu código.

Plane — Complexity: $O(1)$

Usage: 1. `plane::side(pt q)` → Verifica en qué cara o hemisferio respecto al plano infinito se halla el punto q .

Note: Para aquellos infames problemas que obligan a saltar al espacio 3D.

Radial order — Complexity: $O(N \log N)$

Usage: 1. Aplícalo como un comparador estricto para C++ `std::sort` y ordenará los puntos geométricos alrededor del centro barriendo angularmente.

Note: Útil y necesario para construir Envoltentes Convexos (Convex Hulls) y técnicas de "Sweep Line Angular" sin usar el impreciso `atan2`.

Minkowski sum — Complexity: $O(N + M)$

Usage: 1. `minkowski_sum(vector<pt> p, vector<pt> q)` → Agrupa dos figuras sólidas en una sola silueta que representa todos los trazos vectores posibles.

Note: La varita mágica para detectar colisiones o choques exactos entre dos polígonos convexos trasladándose uno hacia el otro.

Delaunay triangulation — Complexity: $O(N \log N)$ esperado

Usage: 1. `delaunay(vector<pt> v)` → Ensambla triángulos sanos, evitando ángulos puntiagudos inútiles, para construir una red entre puntos dispersos.

Note: Es la estructura dual al Diagrama de Voronoi. Evita mandarle puntos encimados o idénticos.

1.5 Strings

KMP — Complexity: $O(N + M)$

Usage: 1. `kmp_pre(string& t)` → Estudia la meta. 2. `kmp(string& s, string& t)` → Busca rápidamente sin repetir pasajes inútiles la existencia del patrón t a lo largo de todo s .

Note: La caja original del String Matching ("Ctrl+F"). Excelente velocidad y bajo consumo de memoria.

Z function — Complexity: $O(N)$

Usage: 1. `z_function(string& s)` → Genera el arreglo Z . Cada celda `z[i]` indica cuántos caracteres del texto a partir de allí son réplicas exactas del prefijo mismo de toda la cadena.

Note: Sumamente poderoso para buscar patrones si unes el patrón al inicio del texto usando un carácter especial ciego (como '#' en el medio).

Manacher — Complexity: $O(N)$

Usage: 1. `manacher(string& s)` → Analiza la cadena insertando delimitadores invisibles para escupir instantáneamente todos los palíndromos alojados sin importar su paridad.

Note: Extrae el palíndromo más largo o cuenta los subpalíndromos centrados en cualquier carácter velozmente.

Suffix array — Complexity: $O(N \log N)$

Usage: 1. `constructSA(string& s)` → Retorna el índice catalogado, ordenado de la A a la Z, de todas las formas posibles de terminar o diseccionar el texto.

Note: Sirve como tu guía telefónica definitiva. Al estar ordenado, puedes usar Búsqueda Binaria sobre el arreglo resultante para buscar palabras parciales velozmente.

LCP array — Complexity: $O(N)$

Usage: 1. `computeLCP(string& s, vector<int>& sa)` → Genera un arreglo de similitudes entre vecinos en tu Suffix Array.

Note: Siempre se usa de la mano con Suffix Array. Convierte los misteriosos problemas de texto en operaciones matemáticas sobre arreglos numéricos (ej. hallar el substring común más largo).

Suffix tree — Complexity: $O(N)$

Usage: 1. `SuffixTree(string s)` → Condensa el abrumador universo de combinaciones del texto en un bello árbol estructurado.

Note: Extremadamente pesado. No olvides rematar el texto con un sello de fin (ej. '\$') para materializar todas las hojas posibles sin problemas invisibles.

String hashing — Complexity: $O(N)$ preálculo, $O(1)$ consulta

Usage: 1. `Hash(string s)` → Fija números de base. 2. `get(int l, int r)` → Otorga el código de identidad inalterable de un pedazo del texto.

Note: Transforma comparaciones molestas de caracteres letra por letra, en triviales comprobaciones `hash_A == hash_B` a velocidad de luz.

1.6 Other

Min-cost max-flow (MCMF) — Complexity: $O(\text{Flujo Max} \cdot E \log V)$

Usage: 1. `add_edge(int u, int v, int cap, int cost)` → Conecta un tubo entre u y v donde pueden pasar hasta `cap` elementos en total, cobrando un peaje o castigo de `cost` por CADA unidad enviada. 2. `get_flow(int s, int t)` → Inicia la simulación en s apuntando a t . Devuelve el par consolidado de {Máximo_Flujo, Mínimo_Costo}.

Note: Envía toda el agua posible pero busca ávidamente las rutas más económicas primero para ahorrar el total. Si tienes aristas iniciales negativas, deberás incluir la limpieza mágica de los "Potenciales" antes del Dijkstra subyacente.

Mo algorithm — Complexity: $O((N + Q)\sqrt{N})$

Usage: 1. `mos()` → Revuelve y ordena las preguntas (consultas) inteligentemente basadas en bloques de raíz cuadrada para anular los saltos extremos largos en el arreglo. 2. `add(pos)` y `remove(pos)` → Al mover la cortina (tu lupa de visualización) un pasito, informas qué pasa con la estadística o el contador total.

Note: Estrictamente "Offline". Útil cuando no puedes usar un Segment Tree porque combinar las respuestas de dos mitades separadas es infernal, pero añadir/quitar un ítem de tu cuenta actual te toma solo $O(1)$.

Divide and conquer DP optimization — Complexity: $O(K \cdot N \log N)$

Usage: 1. `doit(int s, int e, int s0, int e0, int i)` → Corta a la mitad la

zona incierta sabiendo el punto de corte óptimo validado. 2. `doall()` → Orquesta los bucles sobre las iteraciones.

Note: Resuelve misterios de Programación Dinámica tipo "Partición de arreglo en K componentes", acuchillando un inaceptable $O(K \cdot N^2)$ para convertirlo en algo muy veloz. Condición obligatoria: tu punto de corte debe ser estrictamente monótono.

Date conversion — Complexity: $O(1)$

Usage: 1. `dateToInt(int y, int m, int d)` → Exprime una fastidiosa fecha del calendario gregoriano en un simple número absoluto (Día Juliano).

Note: ¡Huye de programar meses bisiestos y validaciones a mano! Convierte fechas a enteros, réstalos libremente, y aplícales módulo 7 para develar el día semanal.

2 Data structures

2.1 Segment tree

```

1 #define oper min
2 #define NEUT INF
3 struct STree { // segment tree for min over integers
4     vector<int> st;int n;
5     STree(int n): st(4*n+5,NEUT), n(n) {}
6     void init(int k, int s, int e, int *a){
7         if(s+1==e){st[k]=a[s];return;}
8         int m=(s+e)/2;
9         init(2*k,s,m,a);init(2*k+1,m,e,a);
10        st[k]=oper(st[2*k],st[2*k+1]);
11    }
12    void upd(int k, int s, int e, int p, int v){
13        if(s+1==e){st[k]=v;return;}
14        int m=(s+e)/2;
15        if(p<m)upd(2*k,s,m,p,v);
16        else upd(2*k+1,m,e,p,v);
17        st[k]=oper(st[2*k],st[2*k+1]);
18    }
19    int query(int k, int s, int e, int a, int b){
20        if(s>=b||e<=a)return NEUT;
21        if(s>=a&&e<=b)return st[k];
22        int m=(s+e)/2;
23        return oper(query(2*k,s,m,a,b),query(2*k+1,m,e,a,b));
24    }
25    void init(int *a){init(1,0,n,a);}
26    void upd(int p, int v){upd(1,0,n,p,v);}
27    int query(int a, int b){return query(1,0,n,a,b);}
28 }; // usage: STree rmq(n);rmq.init(x);rmq.upd(i,v);rmq.query(s,e);

```

2.2 Segment tree - Lazy propagation

```

1 struct STree { // example: range sum with range addition
2     vector<int> st,lazy;int n;
3     STree(int n): st(4*n+5,0), lazy(4*n+5,0), n(n) {}
4     void init(int k, int s, int e, int *a){
5         lazy[k]=0; // lazy neutral element
6         if(s+1==e){st[k]=a[s];return;}
7         int m=(s+e)/2;
8         init(2*k,s,m,a);init(2*k+1,m,e,a);

```



```

9     st[k]=st[2*k]+st[2*k+1]; // operation
10 }
11 void push(int k, int s, int e){
12     if(!lazy[k])return; // if neutral, nothing to do
13     st[k]+=(e-s)*lazy[k]; // update st according to lazy
14     if(s+1<e){ // propagate to children
15         lazy[2*k]+=lazy[k];
16         lazy[2*k+1]+=lazy[k];
17     }
18     lazy[k]=0; // clear node lazy
19 }
20 void upd(int k, int s, int e, int a, int b, int v){
21     push(k,s,e);
22     if(s>=b||e<=a)return;
23     if(s>=a&&e<=b){
24         lazy[k]+=v; // accumulate lazy
25         push(k,s,e);return;
26     }
27     int m=(s+e)/2;
28     upd(2*k,s,m,a,b,v);upd(2*k+1,m,e,a,b,v);
29     st[k]=st[2*k]+st[2*k+1]; // operation
30 }
31 int query(int k, int s, int e, int a, int b){
32     if(s>=b||e<=a)return 0; // operation neutral
33     push(k,s,e);
34     if(s>=a&&e<=b)return st[k];
35     int m=(s+e)/2;
36     return query(2*k,s,m,a,b)+query(2*k+1,m,e,a,b); // operation
37 }
38 void init(int *a){init(1,0,n,a);}
39 void upd(int a, int b, int v){upd(1,0,n,a,b,v);}
40 int query(int a, int b){return query(1,0,n,a,b);}
41 }; // usage: STree rmq(n);rmq.init(x);rmq.upd(s,e,v);rmq.query(s,e);

```

2.3 Segment tree - Persistence

```

1 #define oper(a,b) min(a,b)
2 #define NEUT INF
3 struct STree { // persistent segment tree for min over integers
4     vector<int> st, L, R; int n,sz,rt;
5     STree(int n): st(1,NEUT),L(1,0),R(1,0),n(n),rt(0),sz(1){}
6     int new_node(int v, int l=0, int r=0){
7         int ks=sz(st);

```

```

8         st.push_back(v);L.push_back(l);R.push_back(r);
9         return ks;
10    }
11    int init(int s, int e, int *a){ // not necessary in most cases
12        if(s+1==e)return new_node(a[s]);
13        int m=(s+e)/2,l=init(s,m,a),r=init(m,e,a);
14        return new_node(oper(st[l],st[r]),l,r);
15    }
16    int upd(int k, int s, int e, int p, int v){
17        int ks=new_node(st[k],L[k],R[k]);
18        if(s+1==e){st[ks]=v;return ks;}
19        int m=(s+e)/2,ps;
20        if(p<m)ps=upd(L[ks],s,m,p,v),L[ks]=ps;
21        else ps=upd(R[ks],m,e,p,v),R[ks]=ps;
22        st[ks]=oper(st[L[ks]],st[R[ks]]);
23        return ks;
24    }
25    int query(int k, int s, int e, int a, int b){
26        if(e<=a||b<=s)return NEUT;
27        if(a<=s&&e<=b)return st[k];
28        int m=(s+e)/2;
29        return oper(query(L[k],s,m,a,b),query(R[k],m,e,a,b));
30    }
31    int init(int *a){return init(0,n,a);}
32    int upd(int k, int p, int v){return rt=upd(k,0,n,p,v);}
33    int upd(int p, int v){return upd(rt,p,v);} // update on last root
34    int query(int k,int a, int b){return query(k,0,n,a,b);}
35 }; // usage: STree rmq(n);root=rmq.init(x);new_root=rmq.upd(root,i,v);
    rmq.query(root,s,e);

```

2.4 Segment tree - 2D

```

1 int n,m;
2 int a[MAXN][MAXN],st[2*MAXN][2*MAXN];
3 void build(){
4     for (int i = 0; i < n; ++i)for (int j = 0; j < m; ++j)st[i+n][j+m]=a[i][j];
5     for (int i = 0; i < n; ++i)for(int j=m-1;j--j)
6         st[i+n][j]=op(st[i+n][j<<1],st[i+n][j<<1|1]);
7     for(int i=n-1;i--i)for (int j = 0; j < 2*m; ++j)
8         st[i][j]=op(st[i<<1][j],st[i<<1|1][j]);
9 }
10 void upd(int x, int y, int v){

```

```

11 st[x+n][y+m]=v;
12 for(int j=y+m;j>1;j>>=1)st[x+n][j>>1]=op(st[x+n][j],st[x+n][j^1]);
13 for(int i=x+n;i>1;i>>=1)for(int j=y+m;j>>=1)
14     st[i>>1][j]=op(st[i][j],st[i^1][j]);
15 }
16 int query(int x0, int x1, int y0, int y1){
17     int r=NEUT;
18     for(int i0=x0+n,i1=x1+n;i0<i1;i0>>=1,i1>>=1){
19         int t[4],q=0;
20         if(i0&1)t[q++]=i0++;
21         if(i1&1)t[q++]--i1;
22         for (int k = 0; k < q; ++k)for(int j0=y0+m,j1=y1+m;j0<j1;j0>>=1,j1
23             >>=1){
24             if(j0&1)r=op(r,st[t[k]][j0++]);
25             if(j1&1)r=op(r,st[t[k]][--j1]);
26         }
27     }
28     return r;

```

2.5 Sparse table (static RMQ)

```

1 #define oper min
2 int st[K][1<<K];int n; // K such that 2^K>n
3 void st_init(int *a){
4     for (int i = 0; i < n; ++i)st[0][i]=a[i];
5     for (int k = 1; k < K; ++k)for (int i = 0; i < n-(1<<k)+1; ++i)
6         st[k][i]=oper(st[k-1][i],st[k-1][i+(1<<(k-1))]);
7 }
8 int st_query(int s, int e){
9     int k=31-__builtin_clz(e-s);
10    return oper(st[k][s],st[k][e-(1<<k)]);
11 }

```

2.6 Fenwick tree

```

1 int ft[MAXN+1]; // for more dimensions, make ft multi-dimensional
2 void upd(int i0, int v){ // add v to i0th element (0-based)
3     // add extra fors for more dimensions
4     for(int i=i0+1;i<=MAXN;i+=i&-i)ft[i]+=v;
5 }
6 int get(int i0){ // get sum of range [0,i0]
7     int r=0;
8     // add extra fors for more dimensions

```

```

9     for(int i=i0;i;i-=i&-i)r+=ft[i];
10    return r;
11 }
12 int get_sum(int i0, int i1){ // get sum of range [i0,i1] (0-based)
13     return get(i1)-get(i0);
14 }

```

2.7 Wavelet tree

```

1 struct WT {
2     vector<int> wt[1<<20];int n;
3     void init(int k, int s, int e){
4         if(s+1==e)return;
5         wt[k].clear();wt[k].push_back(0);
6         int m=(s+e)/2;
7         init(2*k,s,m);init(2*k+1,m,e);
8     }
9     void add(int k, int s, int e, int v){
10        if(s+1==e)return;
11        int m=(s+e)/2;
12        if(v<m)wt[k].push_back(wt[k].back()),add(2*k,s,m,v);
13        else wt[k].push_back(wt[k].back()+1),add(2*k+1,m,e,v);
14    }
15    int query0(int k, int s, int e, int a, int b, int i){
16        if(s+1==e)return s;
17        int m=(s+e)/2;
18        int q=(b-a)-(wt[k][b]-wt[k][a]);
19        if(i<q)return query0(2*k,s,m,a-wt[k][a],b-wt[k][b],i);
20        else return query0(2*k+1,m,e,wt[k][a],wt[k][b],i-q);
21    }
22    void upd(int k, int s, int e, int i){
23        if(s+1==e)return;
24        int m=(s+e)/2;
25        int v0=wt[k][i+1]-wt[k][i],v1=wt[k][i+2]-wt[k][i+1];
26        if(!v0&&!v1)upd(2*k,s,m,i-wt[k][i]);
27        else if(v0&&v1)upd(2*k+1,m,e,wt[k][i]);
28        else if(v0)wt[k][i+1]--;
29        else wt[k][i+1]++;
30    }
31    void init(int _n){n=_n;init(1,0,n);} // (values in range [0,n))
32    void add(int v){add(1,0,n,v);}
33    int query0(int a, int b, int i){ // ith element in range [a,b]
34        return query0(1,0,n,a,b,i); // (if it was sorted)

```

```

35 }
36 void upd(int i){ // swap positions i,i+1
37     upd(1,0,n,i);
38 }
39 };

```

2.8 STL extended set

```

1 #include<ext/pb_ds/assoc_container.hpp>
2 #include<ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 typedef tree<int,null_type,less<int>,rb_tree_tag,
5     tree_order_statistics_node_update> ordered_set;
6 // find_by_order(i) -> iterator to ith element
6 // order_of_key(k) -> position (int) of lower_bound of k

```

2.9 STL rope

```

1 #include <ext/rope>
2 using namespace __gnu_cxx;
3 rope<int> s;
4 // Sequence with O(log(n)) random access, insert, erase at any position
5 // s.push_back(x);
6 // s.insert(i,r) // insert rope r at position i
7 // s.erase(i,k) // erase subsequence [i,i+k)
8 // s.substr(i,k) // return new rope corresponding to subsequence [i,i+k)
9 // s[i] // access ith element (cannot modify)
10 // s.mutable_reference_at(i) // acces ith element (allows modification)
11 // s.begin() and s.end() are const iterators (use mutable_begin(),
    mutable_end() to allow modification)

```

2.10 Treap (as BST)

```

1 typedef struct item *pitem;
2 struct item {
3     int pr,key,cnt;
4     pitem l,r;
5     item(int key):key(key),pr(rand()),cnt(1),l(0),r(0) {}
6 };
7 int cnt(pitem t){return t?t->cnt:0;}
8 void upd_cnt(pitem t){if(t)t->cnt=cnt(t->l)+cnt(t->r)+1;}
9 void split(pitem t, int key, pitem& l, pitem& r){ // l: < key, r: >= key
10     if(!t)l=r=0;
11     else if(key<t->key)split(t->l,key,l,t->l),r=t;

```

```

12     else split(t->r,key,t->r,r),l=t;
13     upd_cnt(t);
14 }
15 void insert(pitem& t, pitem it){
16     if(!t)t=it;
17     else if(it->pr>t->pr)split(t,it->key,it->l,it->r),t=it;
18     else insert(it->key<t->key?t->l:t->r,it);
19     upd_cnt(t);
20 }
21 void merge(pitem& t, pitem l, pitem r){
22     if(!l||!r)t=l?l:r;
23     else if(l->pr>r->pr)merge(l->r,l->r,r),t=l;
24     else merge(r->l,l,r->l),t=r;
25     upd_cnt(t);
26 }
27 void erase(pitem& t, int key){
28     if(t->key==key)merge(t,t->l,t->r);
29     else erase(key<t->key?t->l:t->r,key);
30     upd_cnt(t);
31 }
32 void unite(pitem &t, pitem l, pitem r){
33     if(!l||!r){t=l?l:r;return;}
34     if(l->pr<r->pr)swap(l,r);
35     pitem p1,p2;split(r,l->key,p1,p2);
36     unite(l->l,l->l,p1);unite(l->r,l->r,p2);
37     t=l;upd_cnt(t);
38 }
39 pitem kth(pitem t, int k){
40     if(!t)return 0;
41     if(k==cnt(t->l))return t;
42     return k<cnt(t->l)?kth(t->l,k):kth(t->r,k-cnt(t->l)-1);
43 }
44 pair<int,int> lb(pitem t, int key){ // position and value of lower_bound
45     if(!t)return {0,1<<30}; // (special value)
46     if(key>t->key){
47         auto w=lb(t->r,key);w.first+=cnt(t->l)+1;return w;
48     }
49     auto w=lb(t->l,key);
50     if(w.first==cnt(t->l))w.second=t->key;
51     return w;
52 }

```

2.11 Treap (implicit key)

```

1 // example that supports range reverse and addition updates, and range
  sum query
2 // (commented parts are specific to this problem)
3 typedef struct item *pitem;
4 struct item {
5     int pr,cnt,val;
6     // int sum; // (parameters for range query)
7     // bool rev;int add; // (parameters for lazy prop)
8     pitem l,r;
9     item(int val): pr(rand()),cnt(1),val(val),l(0),r(0)/*,sum(val),rev(0),
      add(0)*/ {}
10 };
11 void push(pitem it){
12     if(it){
13         /*if(it->rev){
14             swap(it->l,it->r);
15             if(it->l)it->l->rev^=true;
16             if(it->r)it->r->rev^=true;
17             it->rev=false;
18         }
19         it->val+=it->add;it->sum+=it->cnt*it->add;
20         if(it->l)it->l->add+=it->add;
21         if(it->r)it->r->add+=it->add;
22         it->add=0;*/
23     }
24 }
25 int cnt(pitem t){return t?t->cnt:0;}
26 // int sum(pitem t){return t?push(t),t->sum:0;}
27 void upd_cnt(pitem t){
28     if(t){
29         t->cnt=cnt(t->l)+cnt(t->r)+1;
30         // t->sum=t->val+sum(t->l)+sum(t->r);
31     }
32 }
33 void merge(pitem& t, pitem l, pitem r){
34     push(l);push(r);
35     if(!l||!r)t=l?l:r;
36     else if(l->pr>r->pr)merge(l->r,l->r,r),t=l;
37     else merge(r->l,l,r->l),t=r;
38     upd_cnt(t);
39 }

```

```

40 void split(pitem t, pitem& l, pitem& r, int sz){ // sz:desired size of l
41     if(!t){l=r=0;return;}
42     push(t);
43     if(sz<=cnt(t->l))split(t->l,l,t->l,sz),r=t;
44     else split(t->r,t->r,r,sz-1-cnt(t->l)),l=t;
45     upd_cnt(t);
46 }
47 void output(pitem t){ // useful for debugging
48     if(!t)return;
49     push(t);
50     output(t->l);printf("%d",t->val);output(t->r);
51 }
52 // use merge and split for range updates and queries

```

2.12 Treap (with node father)

```

1 // node father is useful to keep track of the chain of each node
2 // alternative: splay tree
3 // IMPORTANT: add pointer f in struct item
4 void merge(pitem& t, pitem l, pitem r){
5     push(l);push(r);
6     if(!l||!r)t=l?l:r;
7     else if(l->pr>r->pr)merge(l->r,l->r,r),l->r->f=t=l;
8     else merge(r->l,l,r->l),r->l->f=t=r;
9     upd_cnt(t);
10 }
11 void split(pitem t, pitem& l, pitem& r, int sz){
12     if(!t){l=r=0;return;}
13     push(t);
14     if(sz<=cnt(t->l)){
15         split(t->l,l,t->l,sz);r=t;
16         if(l)l->f=0;
17         if(t->l)t->l->f=t;
18     }
19     else {
20         split(t->r,t->r,r,sz-1-cnt(t->l));l=t;
21         if(r)r->f=0;
22         if(t->r)t->r->f=t;
23     }
24     upd_cnt(t);
25 }
26 void push_all(pitem t){
27     if(t->f)push_all(t->f);

```

```

28     push(t);
29 }
30 pitem root(pitem t, int& pos){ // get root and position for node t
31     push_all(t);
32     pos=cnt(t->l);
33     while(t->f){
34         pitem f=t->f;
35         if(t==f->r)pos+=cnt(f->l)+1;
36         t=f;
37     }
38     return t;
39 }

```

2.13 Link-Cut tree

```

1  typedef struct item *pitem;
2  struct item {
3      int pr;bool rev;
4      pitem l,r,f,d;
5      item():pr(rand()),l(0),r(0),f(0),d(0),rev(0){}
6  };
7  void push(pitem t){
8      if(t&&t->rev){
9          swap(t->l,t->r);
10         if(t->l)t->l->rev^=1;
11         if(t->r)t->r->rev^=1;
12         t->rev=0;
13     }
14 }
15 void merge(pitem& t, pitem l, pitem r){
16     push(l);push(r);
17     if(!l||!r)t=l?l:r;
18     else if(l->pr>r->pr)merge(l->r,l->r,r),l->r->f=t=l;
19     else merge(r->l,l,r->l),r->l->f=t=r;
20 }
21 void push_all(pitem t){
22     if(t->f)push_all(t->f);
23     push(t);
24 }
25 void split(pitem t, pitem& l, pitem& r){
26     push_all(t);
27     l=t->l;r=t->r;t->l=t->r=0;
28     while(t->f){

```

```

29     pitem f=t->f;t->f=0;
30     if(t==f->l){
31         if(r)r->f=f;
32         f->l=r;r=f;
33     }
34     else {
35         if(l)l->f=f;
36         f->r=l;l=f;
37     }
38     t=f;
39 }
40 if(l)l->f=0;
41 if(r)r->f=0;
42 }
43 pitem path(pitem p){return p->f?path(p->f):p;}
44 pitem tail(pitem p){push(p);return p->r?tail(p->r):p;}
45 pitem expose(pitem p){
46     pitem q,r,t;
47     split(p,q,r);
48     if(q)tail(q)->d=p;
49     merge(p,p,r);
50     while(t=tail(p),t->d){
51         pitem d=t->d;t->d=0;
52         split(d,q,r);
53         if(q)tail(q)->d=d;
54         merge(p,p,d);merge(p,p,r);
55     }
56     return p;
57 }
58 pitem root(pitem v){return tail(expose(v));}
59 void evert(pitem v){expose(v)->rev^=1;v->d=0;}
60 void link(pitem v, pitem w){ // make v son of w
61     evert(v);
62     pitem p=path(v);
63     merge(p,p,expose(w));
64 }
65 void cut(pitem v){ // cut v from its father
66     pitem p,q;
67     expose(v);split(v,p,q);v->d=0;
68 }
69 void cut(pitem v, pitem w){evert(w);cut(v);}

```

2.14 Convex hull trick (static)

```

1 typedef ll tc;
2 struct Line{tc m,h};
3 struct CHT { // for minimum (for maximum just change the sign of lines)
4     vector<Line> c;
5     int pos=0;
6     tc in(Line a, Line b){
7         tc x=b.h-a.h,y=a.m-b.m;
8         return x/y+(x%y?!((x>0)^(y>0)):0); // ==ceil(x/y)
9     }
10    void add(tc m, tc h){ // m's should be non increasing
11        Line l=(Line){m,h};
12        if(c.size()&&m==c.back().m){
13            l.h=min(h,c.back().h);c.pop_back();if(pos)pos--;
14        }
15        while(c.size()>1&&in(c.back(),l)<=in(c[c.size()-2],c.back())){
16            c.pop_back();if(pos)pos--;
17        }
18        c.push_back(l);
19    }
20    inline bool fbin(tc x, int m){return in(c[m],c[m+1])>x;}
21    tc eval(tc x){
22        // O(log n) query:
23        int s=0,e=c.size();
24        while(e-s>1){int m=(s+e)/2;
25            if(fbin(x,m-1))e=m;
26            else s=m;
27        }
28        return c[s].m*x+c[s].h;
29        // O(1) query (for ordered x's):
30        while(pos>0&&fbin(x,pos-1))pos--;
31        while(pos<c.size()-1&&!fbin(x,pos))pos++;
32        return c[pos].m*x+c[pos].h;
33    }
34 };

```

2.15 Convex hull trick (dynamic)

```

1 typedef ll tc;
2 const tc is_query=-(1LL<<62); // special value for query
3 struct Line {
4     tc m,b;

```

```

5     mutable multiset<Line>::iterator it,end;
6     const Line* succ(multiset<Line>::iterator it) const {
7         return (++it==end? NULL : &*it);}
8     bool operator<(const Line& rhs) const {
9         if(rhs.b!=is_query)return m<rhs.m;
10        const Line *s=succ(it);
11        if(!s)return 0;
12        return b-s->b<(s->m-m)*rhs.m;
13    }
14 };
15 struct HullDynamic : public multiset<Line> { // for maximum
16     bool bad(iterator y){
17         iterator z=next(y);
18         if(y==begin()){
19             if(z==end())return false;
20             return y->m==z->m&&y->b<=z->b;
21         }
22         iterator x=prev(y);
23         if(z==end())return y->m==x->m&&y->b<=x->b;
24         return (x->b-y->b)*(z->m-y->m)>=(y->b-z->b)*(y->m-x->m);
25     }
26     iterator next(iterator y){return ++y;}
27     iterator prev(iterator y){return --y;}
28     void add(tc m, tc b){
29         iterator y=insert((Line){m,b});
30         y->it=y->end=end();
31         if(bad(y)){erase(y);return;}
32         while(next(y)!=end()&&bad(next(y)))erase(next(y));
33         while(y!=begin()&&bad(prev(y)))erase(prev(y));
34     }
35     tc eval(tc x){
36         Line l=*lower_bound((Line){x,is_query});
37         return l.m*x+l.b;
38     }
39 };

```

2.16 Li-Chao tree

```

1 // LiChaoTree lct(sz) -> allows queries for all x in range [0,sz)
2 // lct.add(L)         -> add line L to range [0,sz) in O(log(sz))
3 // lct.add(L,st,en)   -> add line L to range [st,en) in O(log^2(sz))
4 // lct.query(x)       -> query maximum L(x) across all current lines in
                        O(log(sz))

```

```

5
6 typedef long long T;
7 T INF=1e18;
8
9 struct Line{
10     T a, b;
11     Line():a(0),b(-INF){}
12     Line(T a, T b):a(a),b(b){}
13     T get(T x){return a*x+b;}
14 };
15
16 struct LiChaoTree{
17     struct Node{
18         Line line;
19         Node *lc,*rc=0;
20     }*root;
21     int sz;
22     LiChaoTree(int sz): root(new Node()), sz(sz){}
23
24     void addRec(Node* &n, int tl, int tr, Line x){
25         if(!n) n=new Node();
26         if(n->line.get(tl)<x.get(tl)) swap(n->line,x);
27         if(n->line.get(tr-1)>=x.get(tr-1)||tl+1==tr) return;
28         int mid=(tl+tr)/2;
29         if(n->line.get(mid)>x.get(mid)){
30             addRec(n->rc,mid,tr,x);
31         }
32         else{
33             swap(n->line,x);
34             addRec(n->lc,tl,mid,x);
35         }
36     }
37
38     void add(Node* &n, int tl, int tr, int l, int r, Line x){
39         if(tr<=l||r<=tl) return;
40         if(!n) n=new Node();
41         if(l<=tl&&tr<=r) return addRec(n,tl,tr,x);
42         int mid=(tl+tr)/2;
43         add(n->lc,tl,mid,l,r,x);
44         add(n->rc,mid,tr,l,r,x);
45     }
46
47     T query(Node* &n, int tl, int tr, int x){

```

```

48         if(!n) return -INF;
49         if(tl+1==tr) return n->line.get(x);
50         T res=n->line.get(x);
51         int mid=(tl+tr)/2;
52         if(x<mid) res=max(res, query(n->lc,tl,mid,x));
53         else res=max(res, query(n->rc,mid,tr,x));
54         return res;
55     }
56
57     void add(Line x){add(root,0,sz,0,sz,x);}
58     void add(Line x, int l, int r){add(root,0,sz,l,r,x);}
59     T query(int x){return query(root,0,sz,x);}
60 };

```

2.17 Gain-cost-set

```

1 // stores pairs (benefit,cost) (erases non-optimal pairs)
2 struct GCS {
3     set<pair<int,int> > s;
4     void add(int g, int c){
5         pair<int,int> x={g,c};
6         auto p=s.lower_bound(x);
7         if(p!=s.end()&&p->second<=x.second)return;
8         if(p!=s.begin()){ // erase pairs with less benefit
9             --p;           // and more cost
10            while(p->second>=x.second){
11                if(p==s.begin()){s.erase(p);break;}
12                s.erase(p--);
13            }
14        }
15        s.insert(x);
16    }
17    int get(int gain){ // min cost for some benefit
18        auto p=s.lower_bound((pair<int,int>){gain,-INF});
19        int r=p==s.end()?INF:p->second;
20        return r;
21    }
22 };

```

2.18 Disjoint intervals

```

1 // stores disjoint intervals as [first, second)
2 struct disjoint_intervals {
3     set<pair<int,int> > s;

```



```

4 void insert(pair<int,int> v){
5     if(v.first>v.second) return;
6     auto at=s.lower_bound(v);auto it=at;
7     if(at!=s.begin()&&(--at)->second>v.first)v.first=at->first,--it;
8     for(;it!=s.end()&&it->first<=v.second;s.erase(it++))
9         v.second=max(v.second,it->second);
10    segs.insert(v);
11 }
12 };

```

3 Graphs

3.1 Topological sort

```

1 vector<int> g[MAXN];int n;
2 vector<int> tsort(){ // lexicographically smallest topological sort
3     vector<int> r;priority_queue<int> q;
4     vector<int> d(2*n,0);
5     for (int i = 0; i < n; ++i)for (int j = 0; j < g[i].size(); ++j)d[g[i]
6         ][j]++;
7     for (int i = 0; i < n; ++i)if(!d[i])q.push(-i);
8     while(!q.empty()){
9         int x=-q.top();q.pop();r.push_back(x);
10        for (int i = 0; i < g[x].size(); ++i){
11            d[g[x][i]]--;
12            if(!d[g[x][i]])q.push(-g[x][i]);
13        }
14    }
15    return r; // if not DAG it will have less than n elements
16 }

```

3.2 Kruskal (+ Union-Find)

```

1 int uf[MAXN];
2 void uf_init(){memset(uf,-1,sizeof(uf));}
3 int uf_find(int x){return uf[x]<0?x:uf[x]=uf_find(uf[x]);}
4 bool uf_join(int x, int y){
5     x=uf_find(x);y=uf_find(y);
6     if(x==y)return false;
7     if(uf[x]>uf[y])swap(x,y);
8     uf[x]+=uf[y];uf[y]=x;
9     return true;
10 }

```

```

11 vector<pair<ll,pair<int,int> > > es; // edges (cost,(u,v))
12 ll kruskal(){ // assumes graph is connected
13     sort(es.begin(),es.end());uf_init();
14     ll r=0;
15     for (int i = 0; i < es.size(); ++i){
16         int x=es[i].second.first,y=es[i].second.second;
17         if(uf_join(x,y))r+=es[i].first; // (x,y,c) belongs to mst
18     }
19     return r; // total cost
20 }

```

3.3 Dijkstra

```

1 vector<pair<int,int> > g[MAXN]; // u->[(v,cost)]
2 ll dist[MAXN];
3 void dijkstra(int x){
4     memset(dist,-1,sizeof(dist));
5     priority_queue<pair<ll,int> > q;
6     dist[x]=0;q.push({0,x});
7     while(!q.empty()){
8         x=q.top().second;ll c=-q.top().first;q.pop();
9         if(dist[x]!=c)continue;
10        for (int i = 0; i < g[x].size(); ++i){
11            int y=g[x][i].first; ll c=g[x][i].second;
12            if(dist[y]<0||dist[x]+c<dist[y])
13                dist[y]=dist[x]+c,q.push({-dist[y],y});
14        }
15    }
16 }

```

3.4 Bellman-Ford

```

1 int n;
2 vector<pair<int,int> > g[MAXN]; // u->[(v,cost)]
3 ll dist[MAXN];
4 void bford(int src){ // 0(nm)
5     fill(dist,dist+n,INF);dist[src]=0;
6     for (int _ = 0; _ < n; ++_)for (int x = 0; x < n; ++x)if(dist[x]!=INF)
7         for(auto t:g[x]){
8             dist[t.first]=min(dist[t.first],dist[x]+t.second);
9         }
10    for (int x = 0; x < n; ++x)if(dist[x]!=INF)for(auto t:g[x]){
11        if(dist[t.first]>dist[x]+t.second){
12            // neg cycle: all nodes reachable from t.first have -INF distance
13        }
14    }
15 }

```



```

12 // to reconstruct neg cycle: save "prev" of each node, go up from
    t.first until repeating a node. this node and all nodes
    between the two occurrences form a neg cycle
13 }
14 }
15 }

```

3.5 Floyd-Warshall

```

1 // g[i][j]: weight of edge (i, j) or INF if there's no edge
2 // g[i][i]=0
3 ll g[MAXN][MAXN]; int n;
4 void floyd(){ // O(n^3) . Replaces g with min distances
5     for (int k = 0; k < n; ++k) for (int i = 0; i < n; ++i) if (g[i][k] < INF)
6         for (int j = 0; j < n; ++j) if (g[k][j] < INF)
7             g[i][j] = min(g[i][j], g[i][k] + g[k][j]);
8 }
9 bool inNegCycle(int v){ return g[v][v] < 0; }
10 bool hasNegCycle(int a, int b){ // true iff there's neg cycle in between
11     for (int i = 0; i < n; ++i) if (g[a][i] < INF && g[i][b] < INF && g[i][i] < 0)
12         return true;
13     return false;
14 }

```

3.6 Strongly connected components (+ 2-SAT)

```

1 // MAXN: max number of nodes or 2 * max number of variables (2SAT)
2 bool truth[MAXN]; // truth[cmp[i]] = value of variable i (2SAT)
3 int nvar; int neg(int x){ return MAXN - 1 - x; } // (2SAT)
4 vector<int> g[MAXN];
5 int n, lw[MAXN], idx[MAXN], qidx, cmp[MAXN], qcmp;
6 stack<int> st;
7 void tjn(int u){
8     lw[u] = idx[u] = ++qidx;
9     st.push(u); cmp[u] = -2;
10    for (int v: g[u]){
11        if (!idx[v] || cmp[v] == -2){
12            if (!idx[v]) tjn(v);
13            lw[u] = min(lw[u], lw[v]);
14        }
15    }
16    if (lw[u] == idx[u]){
17        int x, l = -1;
18        do { x = st.top(); st.pop(); cmp[x] = qcmp; if (min(x, neg(x)) < nvar) l = x; }

```

```

19    while (x != u);
20    if (l != -1) truth[qcmp] = (cmp[neg(l)] < 0); // (2SAT)
21    qcmp++;
22 }
23 }
24 void scc(){
25     memset(idx, 0, sizeof(idx)); qidx = 0;
26     memset(cmp, -1, sizeof(cmp)); qcmp = 0;
27     for (int i = 0; i < n; ++i) if (!idx[i]) tjn(i);
28 }
29 // Only for 2SAT:
30 void addor(int a, int b){ g[neg(a)].push_back(b); g[neg(b)].push_back(a); }
31 bool satisf(int nvar){
32     nvar = nvar; n = MAXN; scc();
33     for (int i = 0; i < nvar; ++i) if (cmp[i] == cmp[neg(i)]) return false;
34     return true;
35 }

```

3.7 Articulation - Bridges - Biconnected

```

1 vector<int> g[MAXN]; int n;
2 struct edge { int u, v, comp; bool bridge; };
3 vector<edge> e;
4 void add_edge(int u, int v){
5     g[u].push_back(e.size()); g[v].push_back(e.size());
6     e.push_back({u, v, -1, false});
7 }
8 int D[MAXN], B[MAXN], T;
9 int nbc; // number of biconnected components
10 int art[MAXN]; // articulation point iff != 0
11 stack<int> st; // only for biconnected
12 void dfs(int u, int pe){
13     B[u] = D[u] = T++;
14     for (int ne: g[u]) if (ne != pe){
15         int v = e[ne].u ^ e[ne].v ^ u;
16         if (D[v] < 0){
17             st.push(ne); dfs(v, ne);
18             if (B[v] > D[u]) e[ne].bridge = true; // bridge
19             if (B[v] >= D[u]){
20                 art[u]++; // articulation
21                 int last; // start biconnected
22                 do {
23                     last = st.top(); st.pop();

```

```

24     e[last].comp=nbc;
25     } while(last!=ne);
26     nbc++;    // end biconnected
27 }
28 B[u]=min(B[u],B[v]);
29 }
30 else if(D[v]<D[u])st.push(ne),B[u]=min(B[u],D[v]);
31 }
32 }
33 void doit(){
34     memset(D,-1,sizeof(D));memset(art,0,sizeof(art));
35     nbc=T=0;
36     for (int i = 0; i < n; ++i)if(D[i]<0)dfs(i,-1),art[i]--;
37 }

```

3.8 Chu-Liu (minimum spanning arborescence)

```

1 //O(n*m) minimum spanning tree in directed graph
2 //returns -1 if not possible
3 //included i-th edge if take[i]!=0
4 typedef int tw; tw INF=1ll<<30;
5 struct edge{int u,v,id;tw len;};
6 struct ChuLiu{
7     int n; vector<edge> e;
8     vector<int> inc,dec,take,pre,num,id,vis;
9     vector<tw> inw;
10    void add_edge(int x, int y, tw w){
11        inc.push_back(0); dec.push_back(0); take.push_back(0);
12        e.push_back({x,y,SZ(e),w});
13    }
14    ChuLiu(int n):n(n),pre(n),num(n),id(n),vis(n),inw(n){}
15    tw doit(int root){
16        auto e2=e;
17        tw ans=0; int eg=SZ(e)-1,pos=SZ(e)-1;
18        while(1){
19            for (int i = 0; i < n; ++i) inw[i]=INF,id[i]=vis[i]=-1;
20            for(auto ed:e2) if(ed.len<inw[ed.v]){
21                inw[ed.v]=ed.len; pre[ed.v]=ed.u;
22                num[ed.v]=ed.id;
23            }
24            inw[root]=0;
25            for (int i = 0; i < n; ++i) if(inw[i]==INF) return -1;
26            int tot=-1;

```

```

27         for (int i = 0; i < n; ++i){
28             ans+=inw[i];
29             if(i!=root)take[num[i]]++;
30             int j=i;
31             while(vis[j]!=i&&j!=root&&id[j]<0)vis[j]=i,j=pre[j];
32             if(j!=root&&id[j]<0){
33                 id[j]=++tot;
34                 for(int k=pre[j];k!=j;k=pre[k]) id[k]=tot;
35             }
36         }
37         if(tot<0)break;
38         for (int i = 0; i < n; ++i) if(id[i]<0)id[i]=++tot;
39         n=tot+1; int j=0;
40         for (int i = 0; i < SZ(e2); ++i){
41             int v=e2[i].v;
42             e2[j].v=id[e2[i].v];
43             e2[j].u=id[e2[i].u];
44             if(e2[j].v!=e2[j].u){
45                 e2[j].len=e2[i].len-inw[v];
46                 inc.push_back(e2[i].id);
47                 dec.push_back(num[v]);
48                 take.push_back(0);
49                 e2[j++].id=++pos;
50             }
51         }
52         e2.resize(j);
53         root=id[root];
54     }
55     while(pos>eg){
56         if(take[pos]>0) take[inc[pos]]++, take[dec[pos]]--;
57         pos--;
58     }
59     return ans;
60 }
61 };

```

3.9 LCA - Binary Lifting

```

1 vector<int> g[1<<K];int n;    // K such that 2^K>=n
2 int F[K][1<<K],D[1<<K];
3 void lca_dfs(int x){
4     for (int i = 0; i < g[x].size(); ++i){
5         int y=g[x][i];if(y==F[0][x])continue;

```

```

6   F[0][y]=x;D[y]=D[x]+1;lca_dfs(y);
7   }
8   }
9   void lca_init(){
10    D[0]=0;F[0][0]=-1;
11    lca_dfs(0);
12    for (int k = 1; k < K; ++k)for (int x = 0; x < n; ++x)
13        if(F[k-1][x]<0)F[k][x]=-1;
14        else F[k][x]=F[k-1][F[k-1][x]];
15   }
16   int lca(int x, int y){
17       if(D[x]<D[y])swap(x,y);
18       for(int k=K-1;k>=0;--k)if(D[x]-(1<<k)>=D[y])x=F[k][x];
19       if(x==y)return x;
20       for(int k=K-1;k>=0;--k)if(F[k][x]!=F[k][y])x=F[k][x],y=F[k][y];
21       return F[0][x];
22   }

```

3.10 Heavy-Light decomposition

```

1   vector<int> g[MAXN];
2   int wg[MAXN],dad[MAXN],dep[MAXN]; // weight,father,depth
3   void dfs1(int x){
4       wg[x]=1;
5       for(int y:g[x])if(y!=dad[x]){
6           dad[y]=x;dep[y]=dep[x]+1;dfs1(y);
7           wg[x]+=wg[y];
8       }
9   }
10  int curpos,pos[MAXN],head[MAXN];
11  void hld(int x, int c){
12      if(c<0)c=x;
13      pos[x]=curpos++;head[x]=c;
14      int mx=-1;
15      for(int y:g[x])if(y!=dad[x]&&(mx<0||wg[mx]<wg[y]))mx=y;
16      if(mx>=0)hld(mx,c);
17      for(int y:g[x])if(y!=mx&&y!=dad[x])hld(y,-1);
18  }
19  void hld_init(){dad[0]=-1;dep[0]=0;dfs1(0);curpos=0;hld(0,-1);}
20  int query(int x, int y, STree& rmq){
21      int r=NEUT;
22      while(head[x]!=head[y]){
23          if(dep[head[x]]>dep[head[y]])swap(x,y);

```

```

24      r=oper(r,rmq.query(pos[head[y]],pos[y]+1));
25      y=dad[head[y]];
26  }
27  if(dep[x]>dep[y])swap(x,y); // now x is lca
28  r=oper(r,rmq.query(pos[x],pos[y]+1));
29  return r;
30 }
31 // for updating: rmq.upd(pos[x],v);
32 // queries on edges: - assign values of edges to "child" node
33 //                   - change pos[x] to pos[x]+1 in query (line 28)

```

3.11 Centroid decomposition

```

1   vector<int> g[MAXN];int n;
2   bool tk[MAXN];
3   int fat[MAXN]; // father in centroid decomposition
4   int szt[MAXN]; // size of subtree
5   int calcsz(int x, int f){
6       szt[x]=1;
7       for(auto y:g[x])if(y!=f&&!tk[y])szt[x]+=calcsz(y,x);
8       return szt[x];
9   }
10  void cdfs(int x=0, int f=-1, int sz=-1){ // 0(nlogn)
11      if(sz<0)sz=calcsz(x,-1);
12      for(auto y:g[x])if(!tk[y]&&szt[y]*2>=sz){
13          szt[x]=0;cdfs(y,f,sz);return;
14      }
15      tk[x]=true;fat[x]=f;
16      for(auto y:g[x])if(!tk[y])cdfs(y,x);
17  }
18  void centroid(){memset(tk,false,sizeof(tk));cdfs();}

```

3.12 Parallel DFS

```

1   struct Tree {
2       int n,z[2];
3       vector<vector<int>>> g;
4       vector<int> ex,ey,p,w,f,v[2];
5       Tree(int n):g(n),w(n),f(n){}
6       void add_edge(int x, int y){
7           p.push_back(g[x].size());g[x].push_back(ex.size());ex.push_back(x);
8           ey.push_back(y);
9           p.push_back(g[y].size());g[y].push_back(ex.size());ex.push_back(y);
10          ey.push_back(x);

```

```

9   }
10  bool go(int k){ // returns true if it finds new node
11      int& x=z[k];
12      while(x>=0&&
13          (w[x]==g[x].size()||w[x]==g[x].size()-1&&(g[x].back()^1)==f[x]))
14          x=f[x]>=0?ex[f[x]]:-1;
15      if(x<0)return false;
16      if((g[x][w[x]]^1)==f[x])w[x]++;
17      int e=g[x][w[x]],y=ey[e];
18      f[y]=e;w[x]++;w[y]=0;x=y;
19      v[k].push_back(x);
20      return true;
21  }
22  vector<int> erase_edge(int e){
23      e*=2; // erases eth edge, returns smaller component
24      int x=ex[e],y=ey[e];
25      p[g[x].back()]=p[e];
26      g[x][p[e]]=g[x].back();g[x].pop_back();
27      p[g[y].back()]=p[e^1];
28      g[y][p[e^1]]=g[y].back();g[y].pop_back();
29      f[x]=f[y]=-1;
30      w[x]=w[y]=0;
31      z[0]=x;z[1]=y;
32      v[0]={x};v[1]={y};
33      bool d0=true,d1=true;
34      while(d0&&d1)d0=go(0),d1=go(1);
35      if(d1)return v[0];
36      return v[1];
37  }
38  };

```

3.13 Eulerian path

```

1  // Directed version (uncomment commented code for undirected)
2  struct edge {
3      int y;
4      // list<edge>::iterator rev;
5      edge(int y):y(y){}
6  };
7  list<edge> g[MAXN];
8  void add_edge(int a, int b){
9      g[a].push_front(edge(b));//auto ia=g[a].begin();
10     // g[b].push_front(edge(a));auto ib=g[b].begin();

```

```

11     // ia->rev=ib;ib->rev=ia;
12 }
13 vector<int> p;
14 void go(int x){
15     while(g[x].size()){
16         int y=g[x].front().y;
17         //g[y].erase(g[x].front().rev);
18         g[x].pop_front();
19         go(y);
20     }
21     p.push_back(x);
22 }
23 vector<int> get_path(int x){ // get a path that begins in x
24     // check that a path exists from x before calling to get_path!
25     p.clear();go(x);reverse(p.begin(),p.end());
26     return p;
27 }

```

3.14 Dynamic connectivity

```

1  struct UnionFind {
2      int n,comp;
3      vector<int> uf,si,c;
4      UnionFind(int n=0):n(n),comp(n),uf(n),si(n,1){
5          for (int i = 0; i < n; ++i)uf[i]=i;}
6      int find(int x){return x==uf[x]?x:find(uf[x]);}
7      bool join(int x, int y){
8          if((x=find(x))==y=find(y))return false;
9          if(si[x]<si[y])swap(x,y);
10         si[x]+=si[y];uf[y]=x;comp--;c.push_back(y);
11         return true;
12     }
13     int snap(){return c.size();}
14     void rollback(int snap){
15         while(c.size()>snap){
16             int x=c.back();c.pop_back();
17             si[uf[x]]-=si[x];uf[x]=x;comp++;
18         }
19     }
20 };
21 enum {ADD,DEL,QUERY};
22 struct Query {int type,x,y};
23 struct DynCon {

```

```

24 vector<Query> q;
25 UnionFind dsu;
26 vector<int> mt;
27 map<pair<int,int>,int> last;
28 DynCon(int n):dsu(n){
29 void add(int x, int y){
30     if(x>y)swap(x,y);
31     q.push_back((Query){ADD,x,y});mt.push_back(-1);last[{x,y}]=q.size()
32     -1;
33 }
34 void remove(int x, int y){
35     if(x>y)swap(x,y);
36     q.push_back((Query){DEL,x,y});
37     int pr=last[{x,y}];mt[pr]=q.size()-1;mt.push_back(pr);
38 }
39 void query(){q.push_back((Query){QUERY,-1,-1});mt.push_back(-1);}
40 void process(){ // answers all queries in order
41     if(!q.size())return;
42     for (int i = 0; i < q.size(); ++i)if(q[i].type==ADD&&mt[i]<0)mt[i]=q
43     .size();
44     go(0,q.size());
45 }
46 void go(int s, int e){
47     if(s+1==e){
48         if(q[s].type==QUERY) // answer query using DSU
49             printf("%d\n",dsu.comp);
50         return;
51     }
52     int k=dsu.snap(),m=(s+e)/2;
53     for(int i=e-1;i>=m;--i)if(mt[i]>=0&&mt[i]<s)dsu.join(q[i].x,q[i].y);
54     go(s,m);dsu.rollback(k);
55     for(int i=m-1;i>=s;--i)if(mt[i]>=e)dsu.join(q[i].x,q[i].y);
56     go(m,e);dsu.rollback(k);
57 }
58 };

```

3.15 Dominator tree

```

1 //idom[i]=parent of i in dominator tree with root=rt, or -1 if not
  exists
2 int n,rnk[MAXN],pre[MAXN],anc[MAXN],idom[MAXN],semi[MAXN],low[MAXN];
3 vector<int> g[MAXN],rev[MAXN],dom[MAXN],ord;
4 void dfspre(int pos){

```

```

5     rnk[pos]=SZ(ord); ord.push_back(pos);
6     for(auto x:g[pos]){
7         rev[x].push_back(pos);
8         if(rnk[x]==n) pre[x]=pos,dfspre(x);
9     }
10 }
11 int eval(int v){
12     if(anc[v]<n&&anc[anc[v]]<n){
13         int x=eval(anc[v]);
14         if(rnk[semi[low[v]]]>rnk[semi[x]]) low[v]=x;
15         anc[v]=anc[anc[v]];
16     }
17     return low[v];
18 }
19 void dominators(int rt){
20     for (int i = 0; i < n; ++i){
21         dom[i].clear(); rev[i].clear();
22         rnk[i]=pre[i]=anc[i]=idom[i]=n;
23         semi[i]=low[i]=i;
24     }
25     ord.clear(); dfspre(rt);
26     for(int i=SZ(ord)-1;i;i--){
27         int w=ord[i];
28         for(int v:rev[w]){
29             int u=eval(v);
30             if(rnk[semi[w]]>rnk[semi[u]])semi[w]=semi[u];
31         }
32         dom[semi[w]].push_back(w); anc[w]=pre[w];
33         for(int v:dom[pre[w]]){
34             int u=eval(v);
35             idom[v]=(rnk[pre[w]]>rnk[semi[u]]?u:pre[w]);
36         }
37         dom[pre[w]].clear();
38     }
39     for(int w:ord) if(w!=rt&&idom[w]!=semi[w]) idom[w]=idom[idom[w]];
40     for (int i = 0; i < n; ++i) if(idom[i]==n)idom[i]=-1;
41 }

```

3.16 Edmond's blossom (matching in general graphs)

```

1 vector<int> g[MAXN];
2 int n,m,mt[MAXN],qh,qt,q[MAXN],ft[MAXN],bs[MAXN];
3 bool inq[MAXN],inb[MAXN],inp[MAXN];

```

```

4  int lca(int root, int x, int y){
5      memset(inp,0,sizeof(inp));
6      while(1){
7          inp[x=bs[x]]=true;
8          if(x==root)break;
9          x=ft[mt[x]];
10     }
11     while(1){
12         if(inp[y=bs[y]])return y;
13         else y=ft[mt[y]];
14     }
15 }
16 void mark(int z, int x){
17     while(bs[x]!=z){
18         int y=mt[x];
19         inb[bs[x]]=inb[bs[y]]=true;
20         x=ft[y];
21         if(bs[x]!=z)ft[x]=y;
22     }
23 }
24 void contr(int s, int x, int y){
25     int z=lca(s,x,y);
26     memset(inb,0,sizeof(inb));
27     mark(z,x);mark(z,y);
28     if(bs[x]!=z)ft[x]=y;
29     if(bs[y]!=z)ft[y]=x;
30     for (int x = 0; x < n; ++x)if(inb[bs[x]]){
31         bs[x]=z;
32         if(!inq[x])inq[q[++qt]=x]=true;
33     }
34 }
35 int findp(int s){
36     memset(inq,0,sizeof(inq));
37     memset(ft,-1,sizeof(ft));
38     for (int i = 0; i < n; ++i)bs[i]=i;
39     inq[q[qh=qt=0]=s]=true;
40     while(qh<=qt){
41         int x=q[qh++];
42         for(int y:g[x])if(bs[x]!=bs[y]&&mt[x]!=y){
43             if(y==s||mt[y]>=0&&ft[mt[y]]>=0)contr(s,x,y);
44             else if(ft[y]<0){
45                 ft[y]=x;
46                 if(mt[y]<0)return y;

```

```

47         else if(!inq[mt[y]])inq[q[++qt]=mt[y]]=true;
48     }
49 }
50 }
51 return -1;
52 }
53 int aug(int s, int t){
54     int x=t,y,z;
55     while(x>=0){
56         y=ft[x];
57         z=mt[y];
58         mt[y]=x;mt[x]=y;
59         x=z;
60     }
61     return t>=0;
62 }
63 int edmonds(){ // O(n^2 m)
64     int r=0;
65     memset(mt,-1,sizeof(mt));
66     for (int x = 0; x < n; ++x)if(mt[x]<0)r+=aug(x,findp(x));
67     return r;
68 }

```

4 Math

4.1 Identities

$$C_n = \frac{2(2n-1)}{n+1}C_{n-1}$$

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

$$C_n \sim \frac{4^n}{n^{3/2}\sqrt{\pi}}$$

$$\sigma(n) = O(\log(\log(n))) \text{ (number of divisors of } n)$$

$$F_{2n+1} = F_n^2 + F_{n+1}^2$$

$$F_{2n} = F_{n+1}^2 - F_{n-1}^2$$

$$\sum_{i=1}^n F_i = F_{n+2} - 1$$

$$F_{n+i}F_{n+j} - F_nF_{n+i+j} = (-1)^n F_i F_j$$

(Möbius Inv. Formula) Let $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} dg(d)\mu\left(\frac{n}{d}\right)$.

4.2 Theorems

- 1 (Tutte) A graph, $G = (V, E)$, has a perfect matching **if and only if** for every subset U of V , the subgraph induced by $V - U$ has at most $|U|$ connected components with an odd number of vertices.

2 Petersens Theorem. Every cubic, bridgeless graph contains a perfect matching.

3 (Dilworth) In any finite partially ordered set, the maximum number of elements in any antichain equals the minimum number of chains in any partition of the set into chains

4 Pick: $A = I + B/2 - 1$ (area of polygon, points inside, points on border)

4.3 Integer floor division

```
1 void floordiv(ll x, ll y, ll& q, ll& r) { // (for negative x)
2   q=x/y;r=x%y;
3   if((r!=0)&&((r<0)!=(y<0)))q--,r+=y;
4 }
```

4.4 Sieve of Eratosthenes

```
1 int cr[MAXN]; // -1 if prime, some not trivial divisor if not
2 void init_sieve(){
3   memset(cr,-1,sizeof(cr));
4   for (int i = 2; i < MAXN; ++i)if(cr[i]<0)for(ll j=1LL*i*i;j<MAXN;j+=i)
5     cr[j]=i;
6 }
7 map<int,int> fact(int n){ // must call init_crie before
8   map<int,int> r;
9   while(cr[n]>=0)r[cr[n]]++,n/=cr[n];
10  if(n>1)r[n]++;
11  return r;
}
```

4.5 Generate divisors

```
1 void div_rec(vector<ll>& r, vector<pair<ll,int> >& f, int k, ll c){
2   if(k==f.size()){r.push_back(c);return;}
3   for (int i = 0; i < f[k].second+1; ++i)div_rec(r,f,k+1,c),c*=f[k].first;
4 }
5 vector<ll> divisors(vector<pair<ll,int> > f){
6   vector<ll> r; // returns divisors given factorization
7   div_rec(r,f,0,1);
8   return r;
9 }
```

4.6 Pollard's rho

```
1 ll gcd(ll a, ll b){return a?gcd(b%a,a):b;}
2 ll mulmod(ll a, ll b, ll m) {
3   ll r=a*b-(ll)((long double)a*b/m+.5)*m;
4   return r<0?r+m:r;
5 }
6 ll expmod(ll b, ll e, ll m){
7   if(!e)return 1;
8   ll q=expmod(b,e/2,m);q=mulmod(q,q,m);
9   return e&1?mulmod(b,q,m):q;
10 }
11 bool is_prime_prob(ll n, int a){
12   if(n==a)return true;
13   ll s=0,d=n-1;
14   while(d%2==0)s++,d/=2;
15   ll x=expmod(a,d,n);
16   if((x==1)|| (x+1==n))return true;
17   for (int _ = 0; _ < s-1; ++_){
18     x=mulmod(x,x,n);
19     if(x==1)return false;
20     if(x+1==n)return true;
21   }
22   return false;
23 }
24 bool rabin(ll n){ // true iff n is prime
25   if(n==1)return false;
26   int ar[]={2,3,5,7,11,13,17,19,23};
27   for (int i = 0; i < 9; ++i)if(!is_prime_prob(n,ar[i]))return false;
28   return true;
29 }
30 ll rho(ll n){
31   if(!(n&1))return 2;
32   ll x=2,y=2,d=1;
33   ll c=rand()%n+1;
34   while(d==1){
35     x=(mulmod(x,x,n)+c)%n;
36     y=(mulmod(y,y,n)+c)%n;
37     y=(mulmod(y,y,n)+c)%n;
38     if(x>=y)d=gcd(x-y,n);
39     else d=gcd(y-x,n);
40   }
41   return d==n?rho(n):d;
42 }
43 void fact(ll n, map<ll,int>& f){ //O (lg n)^3
```



```

44  if(n==1)return;
45  if(rabin(n)){f[n]++;return;}
46  ll q=rho(n);fact(q,f);fact(n/q,f);
47  }
48  // optimized version: replace rho and fact with the following:
49  const int MAXP=1e6+1; // sieve size
50  int sv[MAXP]; // sieve
51  ll add(ll a, ll b, ll m){return (a+=b)<m?a-a-m;}
52  ll rho(ll n){
53      static ll s[MAXP];
54      while(1){
55          ll x=rand()%n,y=x,c=rand()%n;
56          ll *px=s,*py=s,v=0,p=1;
57          while(1){
58              *py++=y=add(mulmod(y,y,n),c,n);
59              *py++=y=add(mulmod(y,y,n),c,n);
60              if((x=*px++)==y)break;
61              ll t=p;
62              p=mulmod(p,abs(y-x),n);
63              if(!p)return gcd(t,n);
64              if(++v==26){
65                  if((p=gcd(p,n))>1&&p<n)return p;
66                  v=0;
67              }
68          }
69          if(v&&(p=gcd(p,n))>1&&p<n)return p;
70      }
71  }
72  void init_sv(){
73      for (int i = 2; i < MAXP; ++i)if(!sv[i])for(ll j=i;j<MAXP;j+=i)sv[j]=i;
74  }
75  void fact(ll n, map<ll,int>& f){ // call init_sv first!!!
76      for(auto&& p:f){
77          while(n%p.first==0){
78              p.second++;
79              n/=p.first;
80          }
81      }
82      if(n<MAXP)while(n>1)f[sv[n]]++,n/=sv[n];
83      else if(rabin(n))f[n]++;
84      else {ll q=rho(n);fact(q,f);fact(n/q,f);}
85  }

```

4.7 Simpson's rule

```

1  double integrate(double f(double), double a, double b, int n=10000){
2      double r=0,h=(b-a)/n,fa=f(a),fb;
3      for (int i = 0; i < n; ++i){fb=f(a+h*(i+1));r+=fa+4*f(a+h*(i+0.5))+fb;
4          fa=fb;}
5      return r*h/6.;
6  }

```

4.8 Polynomials

```

1  typedef int tp; // type of polynomial
2  template<class T=tp>
3  struct poly { // poly<> : 1 variable, poly<poly<>>: 2 variables, etc.
4      vector<T> c;
5      T& operator[](int k){return c[k];}
6      poly(vector<T>& c):c(c){}
7      poly(initializer_list<T> c):c(c){}
8      poly(int k):c(k){}
9      poly(){}
10     poly operator+(poly<T> o){
11         int m=c.size(),n=o.c.size();
12         poly res(max(m,n));
13         for (int i = 0; i < m; ++i)res[i]=res[i]+c[i];
14         for (int i = 0; i < n; ++i)res[i]=res[i]+o.c[i];
15         return res;
16     }
17     poly operator*(tp k){
18         poly res(c.size());
19         for (int i = 0; i < c.size(); ++i)res[i]=c[i]*k;
20         return res;
21     }
22     poly operator*(poly o){
23         int m=c.size(),n=o.c.size();
24         poly res(m+n-1);
25         for (int i = 0; i < m; ++i)for (int j = 0; j < n; ++j)res[i+j]=res[i+j]+c[i]*o.c[j];
26         return res;
27     }
28     poly operator-(poly<T> o){return *this+(o*-1);}
29     T operator()(tp v){
30         T sum(0);
31         for(int i=c.size()-1;i>=0;--i)sum=sum*v+c[i];

```



```

32     return sum;
33 }
34 };
35 // example: p(x,y)=2*x^2+3*x*y-y+4
36 // poly<poly<>> p={{4,-1},{0,3},{2}}
37 // printf("%d\n",p(2)(3)) // 27 (p(2,3))
38 set<tp> roots(poly<> p){ // only for integer polynomials
39     set<tp> r;
40     while(!p.c.empty()&&!p.c.back())p.c.pop_back();
41     if(!p(0))r.insert(0);
42     if(p.c.empty())return r;
43     tp a0=0,an=abs(p[p.c.size()-1]);
44     for(int k=0;!a0;a0=abs(p[k++]));
45     vector<tp> ps,qs;
46     for (int i = 1; i < sqrt(a0)+1; ++i)if(a0%i==0)ps.push_back(i),ps.
        push_back(a0/i);
47     for (int i = 1; i < sqrt(an)+1; ++i)if(an%i==0)qs.push_back(i),qs.
        push_back(an/i);
48     for(auto pt:ps)for(auto qt:qs)if(pt%qt==0){
49         tp x=pt/qt;
50         if(!p(x))r.insert(x);
51         if(!p(-x))r.insert(-x);
52     }
53     return r;
54 }
55 pair<poly<>,tp> ruffini(poly<> p, tp r){ // returns pair (result,rem)
56     int n=p.c.size()-1;
57     vector<tp> b(n);
58     b[n-1]=p[n];
59     for(int k=n-2;k>=0;--k)b[k]=p[k+1]+r*b[k+1];
60     return {poly<>(b),p[0]+r*b[0]};
61 }
62 // only for double polynomials
63 pair<poly<>,poly<>> polydiv(poly<> p, poly<> q){ // returns pair (
    result,rem)
64     int n=p.c.size()-q.c.size()+1;
65     vector<tp> b(n);
66     for(int k=n-1;k>=0;--k){
67         b[k]=p.c.back()/q.c.back();
68         for (int i = 0; i < q.c.size(); ++i)p[i+k]-=b[k]*q[i];
69         p.c.pop_back();
70     }
71     while(!p.c.empty()&&abs(p.c.back())<EPS)p.c.pop_back();

```

```

72     return {poly<>(b),p};
73 }
74 // only for double polynomials
75 poly<> interpolate(vector<tp> x, vector<tp> y){ //TODO TEST
76     poly<> q={1},S={0};
77     for(tp a:x)q=poly<>((-a,1))*q;
78     for (int i = 0; i < x.size(); ++i){
79         poly<> Li=ruffini(q,x[i]).first;
80         Li=Li*(1.0/Li(x[i])); // change for int polynomials
81         S=S+Li*y[i];
82     }
83     return S;
84 }

```

4.9 Bairstow

```

1 double pget(poly<>& p, int k){return k<p.c.size()?p[k]:0;}
2 poly<> bairstow(poly<> p){ // returns polynomial of degree 2 that
3     int n=p.c.size()-1; // divides p
4     assert(n>=3&&abs(p.c.back())>EPS);
5     double u=p[n-1]/p[n],v=p[n-2]/p[n];
6     for (int _ = 0; _ < ITER; ++_){
7         auto w=polydiv(p,{v,u,1});
8         poly<> q=w.first,r0=w.second;
9         poly<> r1=polydiv(q,{v,u,1}).second;
10        double c=pget(r0,1),d=pget(r0,0),g=pget(r1,1),h=pget(r1,0);
11        double det=1/(v*g*g+h*(h-u*g)),uu=u;
12        u=-det*(-h*c+g*d);v=-det*(-g*v*c+(g*uu-h)*d);
13
14    }
15    return {v,u,1};
16 }
17 void addr(vector<double>& r, poly<>& p){
18     assert(p.c.size()<=3);
19     if(p.c.size()<=1)return;
20     if(p.c.size()==2)r.push_back(-p[0]/p[1]);
21     if(p.c.size()==3){
22         double a=p[2],b=p[1],c=p[0];
23         double d=b*b-4*a*c;
24         if(d<-0.1)return; // huge epsilon because of bad precision
25         d=d>0?sqrt(d):0;r.push_back((-b-d)/2/a);r.push_back((-b+d)/2/a);
26     }
27 }

```

```

28 vector<double> roots(poly<> p){
29     while(!p.c.empty()&&abs(p.c.back())<EPS)p.c.pop_back();
30     for (int i = 0; i < p.c.size(); ++i)p[i]/=p.c.back();
31     vector<double> r;int n;
32     while((n=p.c.size()-1)>=3){
33         poly<> q=bairstow(p);addr(r,q);
34         p=polydiv(p,q).first;
35         while(p.c.size()>n-1)p.c.pop_back();
36     }
37     addr(r,p);
38     return r;
39 }

```

4.10 Fast Fourier Transform

```

1 // MAXN must be power of 2 !!
2 // MOD-1 needs to be a multiple of MAXN !!
3 // big mod and primitive root for NTT:
4 typedef ll tf;
5 typedef vector<tf> poly;
6 const tf MOD=2305843009255636993,RT=5;
7 // FFT
8 struct CD {
9     double r,i;
10    CD(double r=0, double i=0):r(r),i(i){}
11    double real()const{return r;}
12    void operator/=(const int c){r/=c, i/=c;}
13 };
14 CD operator*(const CD& a, const CD& b){
15     return CD(a.r*b.r-a.i*b.i,a.r*b.i+a.i*b.r);}
16 CD operator+(const CD& a, const CD& b){return CD(a.r+b.r,a.i+b.i);}
17 CD operator-(const CD& a, const CD& b){return CD(a.r-b.r,a.i-b.i);}
18 const double pi=acos(-1.0);
19 // NTT
20 /*
21 struct CD {
22     tf x;
23     CD(tf x):x(x){}
24     CD(){}
25 };
26 CD operator*(const CD& a, const CD& b){return CD(mulmod(a.x,b.x));}
27 CD operator+(const CD& a, const CD& b){return CD(addmod(a.x,b.x));}
28 CD operator-(const CD& a, const CD& b){return CD(submod(a.x,b.x));}

```

```

29 vector<tf> rts(MAXN+9,-1);
30 CD root(int n, bool inv){
31     tf r=rts[n]<0?rts[n]=pm(RT,(MOD-1)/n):rts[n];
32     return CD(inv?pm(r,MOD-2):r);
33 }
34 */
35 CD cp1[MAXN+9],cp2[MAXN+9];
36 int R[MAXN+9];
37 void dft(CD* a, int n, bool inv){
38     for (int i = 0; i < n; ++i)if(R[i]<i)swap(a[R[i]],a[i]);
39     for(int m=2;m<=n;m*=2){
40         double z=2*pi/m*(inv?-1:1); // FFT
41         CD wi=CD(cos(z),sin(z)); // FFT
42         // CD wi=root(m,inv); // NTT
43         for(int j=0;j<n;j+=m){
44             CD w(1);
45             for(int k=j,k2=j+m/2;k2<j+m;k++,k2++){
46                 CD u=a[k];CD v=a[k2]*w;a[k]=u+v;a[k2]=u-v;w=w*wi;
47             }
48         }
49     }
50     if(inv)for (int i = 0; i < n; ++i)a[i]/=n; // FFT
51     //if(inv){ // NTT
52     //    CD z(pm(n,MOD-2)); // pm: modular exponentiation
53     //    for (int i = 0; i < n; ++i)a[i]=a[i]*z;
54     //}
55 }
56 poly multiply(poly& p1, poly& p2){
57     int n=p1.size()+p2.size()+1;
58     int m=1,cnt=0;
59     while(m<=n)m+=m,cnt++;
60     for (int i = 0; i < m; ++i){R[i]=0;for (int j = 0; j < cnt; ++j)R[i]=(
61         R[i]<<1)|((i>>j)&1);}
62     for (int i = 0; i < m; ++i)cp1[i]=0,cp2[i]=0;
63     for (int i = 0; i < p1.size(); ++i)cp1[i]=p1[i];
64     for (int i = 0; i < p2.size(); ++i)cp2[i]=p2[i];
65     dft(cp1,m,false);dft(cp2,m,false);
66     for (int i = 0; i < m; ++i)cp1[i]=cp1[i]*cp2[i];
67     dft(cp1,m,true);
68     poly res;
69     n-=2;
70     for (int i = 0; i < n; ++i)res.push_back((tf)floor(cp1[i].real()+0.5))
71         ; // FFT

```

```

70 //for (int i = 0; i < n; ++i)res.push_back(cp1[i].x); // NTT
71 return res;
72 }

```

4.11 FFT operations

```

1 // Polynomial division: O(n*log(n))
2 // Multi-point polynomial evaluation for arbitrarily large points: O(n*
  log^2(n))
3 // Multi-point polynomial evaluation for all points in [0, MOD): O((n+
  MOD)*log(n+MOD))
4 // Polynomial interpolation: O(n*log^2(n))
5 // Inverse: O(n*log(n))
6 // Log: O(n*log(n))
7 // Exp: O(n*log(n))
8
9 //Works with NTT. For FFT, just replace addmod,submod,mulmod,inv
10 poly add(poly &a, poly &b){
11     int n=SZ(a),m=SZ(b);
12     poly ans(max(n,m));
13     for (int i = 0; i < max(n,m); ++i) ans[i]=addmod(i<SZ(a)?a[i]:0, i<SZ(
      b)?b[i]:0);
14     while(SZ(ans)>1&&!ans.back())ans.pop_back();
15     return ans;
16 }
17
18 // derivative of p
19 poly derivate(poly &p){
20     poly ans(max(1, SZ(p)-1));
21     for (int i = 1; i < SZ(p); ++i) ans[i-1]=mulmod(p[i],i);
22     return ans;
23 }
24
25 // integral of p
26 poly integrate(poly &p){
27     poly ans(SZ(p)+1);
28     for (int i = 0; i < SZ(p); ++i) ans[i+1]=mulmod(p[i], inv(i+1));
29     return ans;
30 }
31
32 // p % (x^n)
33 poly takemod(poly &p, int n){
34     poly res=p;

```

```

35     res.resize(min(SZ(res),n));
36     while(SZ(res)>1&&res.back()==0) res.pop_back();
37     return res;
38 }
39
40 // first d terms of 1/p
41 poly invert(poly &p, int d){
42     assert(p[0]);
43     poly res={inv(p[0])};
44     int sz=1;
45     while(sz<d){
46         sz*=2;
47         poly pre(p.begin(), p.begin()+min(SZ(p),sz));
48         poly cur=multiply(res,pre);
49         for (int i = 0; i < SZ(cur); ++i) cur[i]=submod(0,cur[i]);
50         cur[0]=addmod(cur[0],2);
51         res=multiply(res,cur);
52         res=takemod(res,sz);
53     }
54     res.resize(d);
55     return res;
56 }
57
58 // first d terms of log(p)
59 poly log(poly &p, int d){
60     assert(p[0]==1);
61     poly cur=takemod(p,d);
62     poly a=invert(cur,d), b=derivate(cur);
63     auto res=multiply(a,b);
64     res=takemod(res,d-1);
65     res=integrate(res);
66     return res;
67 }
68
69 // first d terms of exp(p)
70 poly exp(poly &p, int d){
71     assert(!p[0]);
72     poly res={1};
73     int sz=1;
74     while(sz<d){
75         sz*=2;
76         poly lg=log(res, sz), cur(sz);
77         for (int i = 0; i < sz; ++i) cur[i]=submod(i<SZ(p)?p[i]:0, i<SZ(lg)?

```

```

    lg[i]:0);
78   cur[0]=addmod(cur[0],1);
79   res=multiply(res,cur);
80   res=takemod(res, sz);
81 }
82
83 res.resize(d);
84 return res;
85 }
86
87 pair<poly,poly> divslow(poly &a, poly &b){
88   poly q,r=a;
89   while(SZ(r)>=SZ(b)){
90     q.push_back(mulmod(r.back(),inv(b.back())));
91     if(q.back()) for (int i = 0; i < SZ(b); ++i){
92       r[SZ(r)-i-1]=submod(r[SZ(r)-i-1],mulmod(q.back(),b[SZ(b)-i-1]));
93     }
94     r.pop_back();
95   }
96   reverse(ALL(q));
97   return {q,r};
98 }
99
100 pair<poly,poly> divide(poly &a, poly &b){ //returns {quotient,remainder}
101   int m=SZ(a),n=SZ(b),MAGIC=750;
102   if(m<n) return {{0},a};
103   if(min(m-n,n)<MAGIC)return divslow(a,b);
104   poly ap=a; reverse(ALL(ap));
105   poly bp=b; reverse(ALL(bp));
106   bp=invert(bp,m-n+1);
107   poly q=multiply(ap,bp);
108   q.resize(SZ(q)+m-n-SZ(q)+1,0);
109   reverse(ALL(q));
110   poly bq=multiply(b,q);
111   for (int i = 0; i < SZ(bq); ++i) bq[i]=submod(0,bq[i]);
112   poly r=add(a,bq);
113   return {q,r};
114 }
115
116 vector<poly> tree;
117
118 void filltree(vector<tf> &x){
119   int k=SZ(x);

```

```

120   tree.resize(2*k);
121   for (int i = k; i < 2*k; ++i) tree[i]={submod(0,x[i-k]),1};
122   for(int i=k-1;i;i--) tree[i]=multiply(tree[2*i],tree[2*i+1]);
123 }
124
125 // Multi-point polynomial evaluation for arbitrarily large points
126 vector<tf> evaluate(poly &a, vector<tf> &x){
127   filltree(x);
128   int k=SZ(x);
129   vector<poly> ans(2*k);
130   ans[1]=divide(a,tree[1]).second;
131   for (int i = 2; i < 2*k; ++i) ans[i]=divide(ans[i>>1],tree[i]).second;
132   vector<tf> r; for (int i = 0; i < k; ++i) r.push_back(ans[i+k][0]);
133   return r;
134 }
135
136 // Given any number g, perform multi-point polynomial evaluation
137 // for all points of the form g^i, for each i in [0, k)
138 // in O((n+k) * log(n+k))
139 vector<int> chirpTransform(poly& p, int g, int k) {
140   int inv_g=inv(g), n=SZ(p)-1, sz=min(k,MOD-1), gk=1;
141   poly ap(n+1), bp(n+sz);
142
143   for (int i = 0; i < n+sz; ++i){
144     ll exp=1ll*i*(i-1)/2%(MOD-1);
145     if(i<=n) ap[n-i]=mulmod(p[i],fpow(inv_g, exp));
146     bp[i]=fpow(g, exp);
147   }
148
149   poly cp=multiply(ap, bp);
150   vector<int> ans(k);
151
152   for (int i = 0; i < sz; ++i){
153     ll exp=1ll*i*(i-1)/2%(MOD-1);
154     int val=0;
155     if(n+i<SZ(cp)) val=cp[n+i];
156     val=mulmod(val, fpow(inv_g,exp));
157     ans[i]=val;
158     gk=mulmod(gk,g);
159   }
160   for (int i = sz; i < k; ++i) ans[i]=ans[i-MOD+1];
161   return ans;
162 }

```

```

163
164 int get_primitive_root(int p) {
165     if(p==2)return 1;
166     int phi=p-1, n=phi;
167     vector<int> fact;
168     for(int i=2;i*i<=n;i++) if(n%i==0){
169         fact.push_back(i);
170         while(n%i==0) n/=i;
171     }
172     if(n>1) fact.push_back(n);
173
174     for (int res = 2; res < p+1; ++res){
175         int ok=1;
176         for(int f:fact){
177             if(fpow(res,phi/f)==1){
178                 ok=false;
179                 break;
180             }
181         }
182         if(ok)return res;
183     }
184     return -1;
185 }
186
187 // Multi-point polynomial evaluation for all points in [0, MOD)
188 // MOD needs to be a prime number
189 vector<int> evaluate_all_points(poly& p) {
190     int g=get_primitive_root(MOD), gk=1;
191     assert(g!=-1);
192
193     vector<int> ch=chirpTransform(p,g,MOD-1), ans(MOD, p[0]);
194
195     for (int i = 0; i < MOD-1; ++i){
196         ans[gk]=ch[i];
197         gk=mulmod(gk,g);
198     }
199     return ans;
200 }
201
202 poly interpolate(vector<tf> &x, vector<tf> &y){
203     filltree(x);
204     poly p=derivate(tree[1]);
205     int k=SZ(y);

```

```

206     vector<tf> d=evaluate(p,x);
207     vector<poly> intree(2*k);
208     for (int i = k; i < 2*k; ++i) intree[i]={mulmod(y[i-k],inv(d[i-k]))};
209     for(int i=k-1;i;i--){
210         poly p1=multiply(tree[2*i],intree[2*i+1]);
211         poly p2=multiply(tree[2*i+1],intree[2*i]);
212         intree[i]=add(p1,p2);
213     }
214     return intree[1];
215 }

```

4.12 Fast Hadamard Transform

```

1  ll c1[MAXN+9],c2[MAXN+9]; // MAXN must be power of 2 !!
2  void fht(ll* p, int n, bool inv){
3      for(int l=1;2*l<=n;l*=2)for(int i=0;i<n;i+=2*l)for (int j = 0; j < l;
4          ++j){
5          ll u=p[i+j],v=p[i+l+j];
6          if(!inv)p[i+j]=u+v,p[i+l+j]=u-v; // XOR
7          else p[i+j]=(u+v)/2,p[i+l+j]=(u-v)/2;
8          //if(!inv)p[i+j]=v,p[i+l+j]=u+v; // AND
9          //else p[i+j]=-u+v,p[i+l+j]=u;
10         //if(!inv)p[i+j]=u+v,p[i+l+j]=u; // OR
11         //else p[i+j]=v,p[i+l+j]=u-v;
12     }
13     // like polynomial multiplication, but XORing exponents
14     // instead of adding them (also ANDing, ORing)
15     vector<ll> multiply(vector<ll>& p1, vector<ll>& p2){
16         int n=1<<(32-__builtin_clz(max(SZ(p1),SZ(p2))-1));
17         for (int i = 0; i < n; ++i)c1[i]=0,c2[i]=0;
18         for (int i = 0; i < SZ(p1); ++i)c1[i]=p1[i];
19         for (int i = 0; i < SZ(p2); ++i)c2[i]=p2[i];
20         fht(c1,n,false);fht(c2,n,false);
21         for (int i = 0; i < n; ++i)c1[i]*=c2[i];
22         fht(c1,n,true);
23         return vector<ll>(c1,c1+n);
24     }

```

4.13 Karatsuba

```

1  typedef ll tp;
2  #define add(n,s,d,k) for (int i = 0; i < n; ++i)(d)[i]+=(s)[i]*k
3  tp* ini(int n){tp *r=new tp[n];fill(r,r+n,0);return r;}

```

```

4 void karatsura(int n, tp* p, tp* q, tp* r){
5     if(n<=0)return;
6     if(n<35)for (int i = 0; i < n; ++i)for (int j = 0; j < n; ++j)r[i+j]+=
        p[i]*q[j];
7     else {
8         int nac=n/2,nbd=n-n/2;
9         tp *a=p,*b=p+nac,*c=q,*d=q+nac;
10        tp *ab=ini(nbd+1),*cd=ini(nbd+1),*ac=ini(nac*2),*bd=ini(nbd*2);
11        add(nac,a,ab,1);add(nbd,b,ab,1);
12        add(nac,c,cd,1);add(nbd,d,cd,1);
13        karatsura(nac,a,c,ac);karatsura(nbd,b,d,bd);
14        add(nac*2,ac,r+nac,-1);add(nbd*2,bd,r+nac,-1);
15        add(nac*2,ac,r,1);add(nbd*2,bd,r+nac*2,1);
16        karatsura(nbd+1,ab,cd,r+nac);
17        free(ab);free(cd);free(ac);free(bd);
18    }
19 }
20 vector<tp> multiply(vector<tp> p0, vector<tp> p1){
21     int n=max(p0.size(),p1.size());
22     tp *p=ini(n),*q=ini(n),*r=ini(2*n);
23     for (int i = 0; i < p0.size(); ++i)p[i]=p0[i];
24     for (int i = 0; i < p1.size(); ++i)q[i]=p1[i];
25     karatsura(n,p,q,r);
26     vector<tp> rr(r,r+p0.size()+p1.size()-1);
27     free(p);free(q);free(r);
28     return rr;
29 }

```

4.14 Diophantine

```

1 pair<ll,ll> extendedEuclid (ll a, ll b){ //a * x + b * y = gcd(a,b)
2     ll x,y;
3     if (b==0) return {1,0};
4     auto p=extendedEuclid(b,a%b);
5     x=p.second;
6     y=p.first-(a/b)*x;
7     if(a*x+b*y==gcd(a,b)) x=-x, y=-y;
8     return {x,y};
9 }
10 pair<pair<ll,ll>,pair<ll,ll> > diophantine(ll a,ll b, ll r) {
11     //a*x+b*y=r where r is multiple of gcd(a,b);
12     ll d=gcd(a,b);
13     a/=d; b/=d; r/=d;

```

```

14     auto p = extendedEuclid(a,b);
15     p.first*=r; p.second*=r;
16     assert(a*p.first+b*p.second==r);
17     return {p,{-b,a}}; // solutions: p+t*ans.second
18 }

```

4.15 Modular inverse

```

1 ll inv(ll a, ll mod) { //inverse of a modulo mod
2     assert(gcd(a,mod)==1);
3     pl sol = extendedEuclid(a,mod);
4     return ((sol.first%mod)+mod)%mod;
5 }

```

4.16 Chinese remainder theorem

```

1 #define mod(a,m) (((a)%m+m)%m)
2 pair<ll,ll> sol(tuple<ll,ll,ll> c){ //requires inv, diophantine
3     ll a=get<0>(c), x1=get<1>(c), m=get<2>(c), d=gcd(a,m);
4     if(d==1) return {mod(x1*inv(a,m),m), m};
5     else return x1%d ? ii({-1LL,-1LL}) : sol(make_tuple(a/d,x1/d,m/d));
6 }
7 pair<ll,ll> crt(vector< tuple<ll,ll,ll> > cond) { // returns: (sol, lcm)
8     ll x1=0,m1=1,x2,m2;
9     for(auto t:cond){
10         tie(x2,m2)=sol(t);
11         if(m2<0 || (x1-x2)%gcd(m1,m2))return {-1,-1};
12         if(m1==m2)continue;
13         ll k=diophantine(m2,-m1,x1-x2).first.second,l=m1*(m2/gcd(m1,m2));
14         x1=mod((__int128)m1*k+x1,l);m1=l;
15     }
16     return sol(make_tuple(1,x1,m1));
17 } //cond[i]={ai,bi,mi} ai*xi=bi (mi); assumes lcm fits in ll

```

4.17 Mobius

```

1 short mu[MAXN] = {0,1};
2 void mobius(){
3     for (int i = 1; i < MAXN; ++i)if(mu[i])for(int j=i+i;j<MAXN;j+=i)mu[j]
        -=mu[i];
4 }

```

4.18 Matrix exponentiation

```

1 typedef vector<vector<ll> > Matrix;
2 Matrix ones(int n) {
3     Matrix r(n,vector<ll>(n));
4     for (int i = 0; i < n; ++i)r[i][i]=1;
5     return r;
6 }
7 Matrix operator*(Matrix &a, Matrix &b) {
8     int n=SZ(a),m=SZ(b[0]),z=SZ(a[0]);
9     Matrix r(n,vector<ll>(m));
10    for (int i = 0; i < n; ++i)for (int j = 0; j < m; ++j)for (int k = 0;
11        k < z; ++k)
12        r[i][j]+=a[i][k]*b[k][j],r[i][j]%=mod;
13    return r;
14 }
15 Matrix be(Matrix b, ll e) {
16     Matrix r=ones(SZ(b));
17     while(e){if(e&1LL)r=r*b;b=b*b;e/=2;}
18     return r;
19 }

```

4.19 Matrix reduce and determinant

```

1 double reduce(vector<vector<double> >& x){ // returns determinant
2     int n=x.size(),m=x[0].size();
3     int i=0,j=0;double r=1.;
4     while(i<n&&j<m){
5         int l=i;
6         for (int k = i+1; k < n; ++k)if(abs(x[k][j])>abs(x[l][j]))l=k;
7         if(abs(x[l][j])<EPS){j++;r=0.;continue;}
8         if(l!=i){r=-r;swap(x[i],x[l]);}
9         r*=x[i][j];
10        for(int k=m-1;k>=j;k--)x[i][k]/=x[i][j];
11        for (int k = 0; k < n; ++k){
12            if(k==i)continue;
13            for(int l=m-1;l>=j;l--)x[k][l]-=x[k][j]*x[i][l];
14        }
15        i++;j++;
16    }
17    return r;
18 }

```

4.20 Simplex

```

1 vector<int> X,Y;

```

```

2 vector<vector<double> > A;
3 vector<double> b,c;
4 double z;
5 int n,m;
6 void pivot(int x,int y){
7     swap(X[y],Y[x]);
8     b[x]/=A[x][y];
9     for (int i = 0; i < m; ++i)if(i!=y)A[x][i]/=A[x][y];
10    A[x][y]=1/A[x][y];
11    for (int i = 0; i < n; ++i)if(i!=x&&abs(A[i][y])>EPS){
12        b[i]-=A[i][y]*b[x];
13        for (int j = 0; j < m; ++j)if(j!=y)A[i][j]-=A[i][y]*A[x][j];
14        A[i][y]=-A[i][y]*A[x][y];
15    }
16    z+=c[y]*b[x];
17    for (int i = 0; i < m; ++i)if(i!=y)c[i]-=c[y]*A[x][i];
18    c[y]=-c[y]*A[x][y];
19 }
20 pair<double,vector<double> > simplex( // maximize c^T x s.t. Ax<=b, x>=0
21     vector<vector<double> > _A, vector<double> _b, vector<double> _c){
22     // returns pair (maximum value, solution vector)
23     A=_A;b=_b;c=_c;
24     n=b.size();m=c.size();z=0.;
25     X=vector<int>(m);Y=vector<int>(n);
26     for (int i = 0; i < m; ++i)X[i]=i;
27     for (int i = 0; i < n; ++i)Y[i]=i+m;
28     while(1){
29         int x=-1,y=-1;
30         double mn=-EPS;
31         for (int i = 0; i < n; ++i)if(b[i]<mn)mn=b[i],x=i;
32         if(x<0)break;
33         for (int i = 0; i < m; ++i)if(A[x][i]<-EPS){y=i;break;}
34         assert(y>=0); // no solution to Ax<=b
35         pivot(x,y);
36     }
37     while(1){
38         double mx=EPS;
39         int x=-1,y=-1;
40         for (int i = 0; i < m; ++i)if(c[i]>mx)mx=c[i],y=i;
41         if(y<0)break;
42         double mn=1e200;
43         for (int i = 0; i < n; ++i)if(A[i][y]>EPS&&b[i]/A[i][y]<mn)mn=b[i]/A
            [i][y],x=i;

```



```

44     assert(x>=0); // c^T x is unbounded
45     pivot(x,y);
46 }
47 vector<double> r(m);
48 for (int i = 0; i < n; ++i) if(Y[i]<m) r[Y[i]]=b[i];
49 return {z,r};
50 }

```

4.21 Discrete log

```

1 //returns x such that a^x = b (mod m) or -1 if inexistent
2 ll discrete_log(ll a,ll b,ll m) {
3     a%=m, b%=m;
4     if(b == 1) return 0;
5     int cnt=0;
6     ll tmp=1;
7     for(int g=__gcd(a,m); g!=1; g=__gcd(a,m)) {
8         if(b%g) return -1;
9         m/=g, b/=g;
10        tmp = tmp*a/g%m;
11        ++cnt;
12        if(b == tmp) return cnt;
13    }
14    map<ll,int> w;
15    int s = ceil(sqrt(m));
16    ll base = b;
17    for (int i = 0; i < s; ++i) {
18        w[base] = i;
19        base=base*a%m;
20    }
21    base=fastpow(a,s,m);
22    ll key=tmp;
23    for (int i = 1; i < s+2; ++i) {
24        key=base*key%m;
25        if(w.count(key)) return i*s-w[key]+cnt;
26    }
27    return -1;
28 }

```

4.22 Berlekamp Massey

```

1 typedef vector<int> vi;
2 vi BM(vi x){
3     vi ls,cur;int lf,ld;

```

```

4     for (int i = 0; i < SZ(x); ++i){
5         ll t=0;
6         for (int j = 0; j < SZ(cur); ++j) t=(t+x[i-j-1]*(ll)cur[j])%MOD;
7         if((t-x[i])%MOD==0) continue;
8         if(!SZ(cur)){cur.resize(i+1);lf=i;ld=(t-x[i])%MOD;continue;}
9         ll k=-(x[i]-t)*fast_pow(ld,MOD-2)%MOD;
10        vi c(i-lf-1);c.push_back(k);
11        for (int j = 0; j < SZ(ls); ++j) c.push_back(-ls[j]*k%MOD);
12        if(SZ(c)<SZ(cur)) c.resize(SZ(cur));
13        for (int j = 0; j < SZ(cur); ++j) c[j]=(c[j]+cur[j])%MOD;
14        if(i-lf+SZ(ls)>=SZ(cur)) ls=cur,lf=i,ld=(t-x[i])%MOD;
15        cur=c;
16    }
17    for (int i = 0; i < SZ(cur); ++i) cur[i]=(cur[i]%MOD+MOD)%MOD;
18    return cur;
19 }

```

4.23 Linear Rec

```

1 //init O(n^2log) query(n^2 logk)
2 //input: terms: first n term; trans: transition function; MOD; LOG=mxlog
3 //output calc(k): kth term mod MOD
4 //example: {1,1} {2,1} an=2*a_(n-1)+a_(n-2); calc(3)=3 calc(10007)
5 //          =71480733
6 struct LinearRec{
7     typedef vector<int> vi;
8     int n; vi terms, trans; vector<vi> bin;
9     vi add(vi &a, vi &b){
10        vi res(n*2+1);
11        for (int i = 0; i < n+1; ++i) for (int j = 0; j < n+1; ++j) res[i+j]=
12            res[i+j]*1LL+(ll)a[i]*b[j]%MOD;
13        for(int i=2*n; i>n; --i){
14            for (int j = 0; j < n; ++j) res[i-1-j]=(res[i-1-j]*1LL+(ll)res[i]*
15                trans[j])%MOD;
16            res[i]=0;
17        }
18        res.erase(res.begin()+n+1,res.end());
19        return res;
20    }
21    LinearRec(vi &terms, vi &trans):terms(terms),trans(trans){
22        n=SZ(trans);vi a(n+1);a[1]=1;
23        bin.push_back(a);

```



```

21     for (int i = 1; i < LOG; ++i) bin.push_back(add(bin[i-1], bin[i-1]));
22 }
23 int calc(int k){
24     vi a(n+1); a[0]=1;
25     for (int i = 0; i < LOG; ++i) if((k>>i)&1) a=add(a, bin[i]);
26     int ret=0;
27     for (int i = 0; i < n; ++i) ret=((ll)ret+(ll)a[i+1]*terms[i])%MOD;
28     return ret;
29 }
30 };

```

4.24 Tonelli Shanks

```

1 ll legendre(ll a, ll p){
2     if(a%p==0) return 0; if(p==2) return 1;
3     return fpow(a, (p-1)/2, p);
4 }
5 ll tonelli_shanks(ll n, ll p){ // sqrt(n) mod p (p must be a prime)
6     assert(legendre(n,p)==1); if(p==2) return 1;
7     ll s=__builtin_ctzll(p-1), q=(p-1LL)>>s, z=rnd(1,p-1);
8     if(s==1) return fpow(n, (p+1)/4LL, p);
9     while(legendre(z,p)!=p-1) z=rnd(1,p-1);
10    ll c=fpow(z,q,p), r=fpow(n,(q+1)/2,p), t=fpow(n,q,p), m=s;
11    while(t!=1){
12        ll i=1, ts=(t*t)%p;
13        while(ts!=1) i++, ts=(ts*ts)%p;
14        ll b=c;
15        for (int _ = 0; _ < m-i-1; ++_) b=(b*b)%p;
16        r=r*b%p; c=b*b%p; t=t*c%p; m=i;
17    }
18    return r;
19 }

```

4.25 Prime count

```

1 // Count number of primes in [1,n]
2 // O(n^(2/3)) time
3 // O(sqrt(n)) memory
4 ll prime_count(ll n) {
5     if (n <= 1) return 0;
6     int v = sqrtl(n), s = (v+1)>>1, pc=0;
7     vector<int> smalls(s), roughs(s), skip(v+1);
8     vector<ll> larges(s);
9     for (int i = 0; i < s; ++i){

```

```

10         smalls[i]=i;
11         roughs[i]=2*i+1;
12         larges[i]=(n/roughs[i]-1)>>1;
13     }
14     for(int p=3;p<=v;p+=2) if(!skip[p]){
15         int q=p*p;
16         if(1ll*q>n) break;
17         skip[p]=1;
18         for(int i=q;i<=v;i+=2*p) skip[i]=1;
19         int ns=0;
20         for (int k = 0; k < s; ++k){
21             int i=roughs[k];
22             if(skip[i]) continue;
23             ll d=1ll*i*p;
24             larges[ns]=larges[k]-(d<=v?larges[smalls[d]>>1]-pc:smalls[(n
                /d-1)>>1])+pc;
25             roughs[ns++]=i;
26         }
27         s=ns;
28         for(int i=(v-1)>>1, j=((v/p)-1)|1; j>=p; j-=2){
29             int c=smalls[j>>1]-pc;
30             for(int e=(j*p)>>1; i>=e; i--) smalls[i]-=c;
31         }
32         pc++;
33     }
34     larges[0]+=1ll*(s+2*(pc-1))*(s-1)>>1;
35     for (int k = 1; k < s; ++k) larges[0]-=larges[k];
36     for (int l = 1; l < s; ++l){
37         int q=roughs[l];
38         ll m=n/q, t=0;
39         int e=smalls[(m/q-1)>>1]-pc;
40         if(e<1+1) break;
41         for (int k = l+1; k < e+1; ++k) t+=smalls[(m/roughs[k]-1)>>1];
42         larges[0]+=t-1ll*(e-1)*(pc+l-1);
43     }
44     return larges[0]+1;
45 }

```

5 Geometry

5.1 Point

```

1 struct pt { // for 3D add z coordinate

```

```

2 double x,y;
3 pt(double x, double y):x(x),y(y){}
4 pt(){}
5 double norm2(){return *this**this;}
6 double norm(){return sqrt(norm2());}
7 bool operator==(pt p){return abs(x-p.x)<=EPS&&abs(y-p.y)<=EPS;}
8 pt operator+(pt p){return pt(x+p.x,y+p.y);}
9 pt operator-(pt p){return pt(x-p.x,y-p.y);}
10 pt operator*(double t){return pt(x*t,y*t);}
11 pt operator/(double t){return pt(x/t,y/t);}
12 double operator*(pt p){return x*p.x+y*p.y;}
13 // pt operator^(pt p){ // only for 3D
14 // return pt(y*p.z-z*p.y,z*p.x-x*p.z,x*p.y-y*p.x);}
15 double angle(pt p){ // redefine acos for values out of range
16 return acos(*this*p/(norm()*p.norm()));}
17 pt unit(){return *this/norm();}
18 double operator%(pt p){return x*p.y-y*p.x;}
19 // 2D from now on
20 bool operator<(pt p)const{ // for convex hull
21 return x<p.x-EPS|| (abs(x-p.x)<=EPS&&y<p.y-EPS);}
22 bool left(pt p, pt q){ // is it to the left of directed line pq?
23 return (q-p)%(*this-p)>EPS;}
24 pt rot(pt r){return pt(*this*r,*this*r);}
25 pt rot(double a){return rot(pt(sin(a),cos(a)));}
26 };
27 pt ccw90(1,0);
28 pt cw90(-1,0);

```

5.2 Line

```

1 int sgn2(double x){return x<0?-1:1;}
2 struct ln {
3 pt p,pq;
4 ln(pt p, pt q):p(p),pq(q-p){}
5 ln(){}
6 bool has(pt r){return dist(r)<=EPS;}
7 bool seghas(pt r){return has(r)&&(r-p)*(r-(p+pq))<=EPS;}
8 // bool operator/(ln l){return (pq.unit()^l.pq.unit()).norm()<=EPS;}
9 // 3D
10 bool operator/(ln l){return abs(pq.unit()%l.pq.unit())<=EPS;} // 2D
11 bool operator==(ln l){return *this/l&&has(l.p);}
12 pt operator^(ln l){ // intersection
13 if(*this/l)return pt(DINF,DINF);

```

```

13 pt r=l.p+l.pq*((p-l.p)%pq/(l.pq%pq));
14 // if(!has(r)){return pt(NAN,NAN,NAN);} // check only for 3D
15 return r;
16 }
17 double angle(ln l){return pq.angle(l.pq);}
18 int side(pt r){return has(r)?0:sgn2(pq%(r-p));} // 2D
19 pt proj(pt r){return p+pq*((r-p)*pq/pq.norm2());}
20 pt ref(pt r){return proj(r)*2-r;}
21 double dist(pt r){return (r-proj(r)).norm();}
22 // double dist(ln l){ // only 3D
23 // if(*this/l)return dist(l.p);
24 // return abs((l.p-p)*(pq^l.pq))/(pq^l.pq).norm();
25 // }
26 ln rot(auto a){return ln(p,p+pq.rot(a));} // 2D
27 };
28 ln bisector(ln l, ln m){ // angle bisector
29 pt p=l^m;
30 return ln(p,p+l.pq.unit()+m.pq.unit());
31 }
32 ln bisector(pt p, pt q){ // segment bisector (2D)
33 return ln((p+q)*.5,p).rot(ccw90);
34 }

```

5.3 Circle

```

1 struct circle {
2 pt o;double r;
3 circle(pt o, double r):o(o),r(r){}
4 circle(pt x, pt y, pt z){o=bisector(x,y)^bisector(x,z);r=(o-x).norm()
5 ;}
6 bool has(pt p){return (o-p).norm()<=r+EPS;}
7 vector<pt> operator^(circle c){ // ccw
8 vector<pt> s;
9 double d=(o-c.o).norm();
10 if(d>r+c.r+EPS||d+min(r,c.r)+EPS<max(r,c.r))return s;
11 double x=(d*d-c.r*c.r+r*r)/(2*d);
12 double y=sqrt(r*r-x*x);
13 pt v=(c.o-o)/d;
14 s.push_back(o+v*x-v.rot(ccw90)*y);
15 if(y>EPS)s.push_back(o+v*x+v.rot(ccw90)*y);
16 return s;
17 }
18 vector<pt> operator^(ln l){

```

```

18     vector<pt> s;
19     pt p=l.proj(o);
20     double d=(p-o).norm();
21     if(d>EPS)return s;
22     if(abs(d-r)<=EPS){s.push_back(p);return s;}
23     d=sqrt(r*r-d*d);
24     s.push_back(p+l.pq.unit()*d);
25     s.push_back(p-l.pq.unit()*d);
26     return s;
27 }
28 vector<pt> tang(pt p){
29     double d=sqrt((p-o).norm2()-r*r);
30     return *this^circle(p,d);
31 }
32 bool in(circle c){ // non strict
33     double d=(o-c.o).norm();
34     return d+r<=c.r+EPS;
35 }
36 double intertriangle(pt a, pt b){ // area of intersection with oab
37     if(abs((o-a)%(o-b))<=EPS)return 0.;
38     vector<pt> q={a},w=*this^ln(a,b);
39     if(w.size()==2)for(auto p:w)if((a-p)*(b-p)<-EPS)q.push_back(p);
40     q.push_back(b);
41     if(q.size()==4&&(q[0]-q[1])*(q[2]-q[1])>EPS)swap(q[1],q[2]);
42     double s=0;
43     for (int i = 0; i < q.size()-1; ++i){
44         if(!has(q[i])||!has(q[i+1]))s+=r*r*(q[i]-o).angle(q[i+1]-o)/2;
45         else s+=abs((q[i]-o)%(q[i+1]-o)/2);
46     }
47     return s;
48 }
49 };
50 vector<double> intercircles(vector<circle> c){
51     vector<double> r(SZ(c)+1); // r[k]: area covered by at least k circles
52     for (int i = 0; i < SZ(c); ++i){ // 0(n^2 log n) (high
53         constant)
54         int k=1;Cmp s(c[i].o);
55         vector<pair<pt,int> > p={
56             {c[i].o+pt(1,0)*c[i].r,0},
57             {c[i].o-pt(1,0)*c[i].r,0}};
58         for (int j = 0; j < SZ(c); ++j)if(j!=i){
59             bool b0=c[i].in(c[j]),b1=c[j].in(c[i]);
60             if(b0&&(!b1||i<j))k++;

```

```

60     else if(!b0&&!b1){
61         auto v=c[i]^c[j];
62         if(SZ(v)==2){
63             p.push_back({v[0],1});p.push_back({v[1],-1});
64             if(s(v[1],v[0]))k++;
65         }
66     }
67 }
68 sort(p.begin(),p.end(),
69     [&](pair<pt,int> a, pair<pt,int> b){return s(a.first,b.first);});
70 for (int j = 0; j < SZ(p); ++j){
71     pt p0=p[j?j-1:SZ(p)-1].first,p1=p[j].first;
72     double a=(p0-c[i].o).angle(p1-c[i].o);
73     r[k]+=(p0.x-p1.x)*(p0.y+p1.y)/2+c[i].r*c[i].r*(a-sin(a))/2;
74     k+=p[j].second;
75 }
76 }
77 return r;
78 }

```

5.4 Polygon

```

1 int sgn(double x){return x<-EPS?-1:x>EPS;}
2 struct pol {
3     int n;vector<pt> p;
4     pol(){}
5     pol(vector<pt> _p){p=_p;n=p.size();}
6     double area(){
7         double r=0.;
8         for (int i = 0; i < n; ++i)r+=p[i]%p[(i+1)%n];
9         return abs(r)/2; // negative if CW, positive if CCW
10    }
11    pt centroid(){ // (barycenter)
12        pt r(0,0);double t=0;
13        for (int i = 0; i < n; ++i){
14            r=r+(p[i]+p[(i+1)%n])*(p[i]%p[(i+1)%n]);
15            t+=p[i]%p[(i+1)%n];
16        }
17        return r/t/3;
18    }
19    bool has(pt q){ // 0(n)
20        for (int i = 0; i < n; ++i)if(ln(p[i],p[(i+1)%n]).seghas(q))return
21            true;

```

```

21     int cnt=0;
22     for (int i = 0; i < n; ++i){
23         int j=(i+1)%n;
24         int k=sgn((q-p[j])%(p[i]-p[j]));
25         int u=sgn(p[i].y-q.y),v=sgn(p[j].y-q.y);
26         if(k>0&&u<0&&v>=0)cnt++;
27         if(k<0&&v<0&&u>=0)cnt--;
28     }
29     return cnt!=0;
30 }
31 void normalize(){ // (call before haslog, remove collinear first)
32     if(p[2].left(p[0],p[1]))reverse(p.begin(),p.end());
33     int pi=min_element(p.begin(),p.end())-p.begin();
34     vector<pt> s(n);
35     for (int i = 0; i < n; ++i)s[i]=p[(pi+i)%n];
36     p.swap(s);
37 }
38 bool haslog(pt q){ // O(log(n)) only CONVEX. Call normalize first
39     if(q.left(p[0],p[1])||q.left(p.back(),p[0]))return false;
40     int a=1,b=p.size()-1; // returns true if point on boundary
41     while(b-a>1){ // (change sign of EPS in left
42         int c=(a+b)/2; // to return false in such case)
43         if(!q.left(p[0],p[c]))a=c;
44         else b=c;
45     }
46     return !q.left(p[a],p[a+1]);
47 }
48 pt farthest(pt v){ // O(log(n)) only CONVEX
49     if(n<10){
50         int k=0;
51         for (int i = 1; i < n; ++i)if(v*(p[i]-p[k])>EPS)k=i;
52         return p[k];
53     }
54     if(n==SZ(p))p.push_back(p[0]);
55     pt a=p[1]-p[0];
56     int s=0,e=n,ua=v*a>EPS;
57     if(!ua&&v*(p[n-1]-p[0])<=EPS)return p[0];
58     while(1){
59         int m=(s+e)/2;pt c=p[m+1]-p[m];
60         int uc=v*c>EPS;
61         if(!uc&&v*(p[m-1]-p[m])<=EPS)return p[m];
62         if(ua&&(!uc||v*(p[s]-p[m])>EPS))e=m;
63         else if(ua||uc||v*(p[s]-p[m])>=-EPS)s=m,a=c,ua=uc;

```

```

64         else e=m;
65         assert(e>s+1);
66     }
67 }
68 pol cut(ln l){ // cut CONVEX polygon by line l
69     vector<pt> q; // returns part at left of l.pq
70     for (int i = 0; i < n; ++i){
71         int d0=sgn(l.pq%(p[i]-l.p)),d1=sgn(l.pq%(p[(i+1)%n]-l.p));
72         if(d0>=0)q.push_back(p[i]);
73         ln m(p[i],p[(i+1)%n]);
74         if(d0*d1<0&&!(1/m))q.push_back(l^m);
75     }
76     return pol(q);
77 }
78 double intercircle(circle c){ // area of intersection with circle
79     double r=0.;
80     for (int i = 0; i < n; ++i){
81         int j=(i+1)%n;double w=c.intertriangle(p[i],p[j]);
82         if((p[j]-c.o)%(p[i]-c.o)>0)r+=w;
83         else r-=w;
84     }
85     return abs(r);
86 }
87 double callipers(){ // square distance of most distant points
88     double r=0; // prereq: convex, ccw, NO COLLINEAR POINTS
89     for(int i=0,j=n<2?0:1;i<j;++i){
90         for(;;j=(j+1)%n){
91             r=max(r,(p[i]-p[j]).norm2());
92             if((p[(i+1)%n]-p[i])%(p[(j+1)%n]-p[j])<=EPS)break;
93         }
94     }
95     return r;
96 }
97 };
98 // Dynamic convex hull trick
99 vector<pol> w;
100 void add(pt q){ // add(q), O(log^2(n))
101     vector<pt> p={q};
102     while(!w.empty()&&SZ(w.back().p)<2*SZ(p)){
103         for(pt v:w.back().p)p.push_back(v);
104         w.pop_back();
105     }
106     w.push_back(pol(chull(p)));

```

```

107 }
108 ll query(pt v){ // max(q*v:q in w), O(log^2(n))
109     ll r=-INF;
110     for(auto& p:w)r=max(r,p.farthest(v)*v);
111     return r;
112 }

```

5.5 Plane

```

1 struct plane {
2     pt a,n; // n: normal unit vector
3     plane(pt a, pt b, pt c):a(a),n(((b-a)^(c-a)).unit()){
4     plane(){}
5     bool has(pt p){return abs((p-a)*n)<=EPS;}
6     double angle(plane w){return acos(n*w.n);}
7     double dist(pt p){return abs((p-a)*n);}
8     pt proj(pt p){inter(ln(p,p+n),p);return p;}
9     bool inter(ln l, pt& r){
10         double x=n*(l.p+l.pq-a),y=n*(l.p-a);
11         if(abs(x-y)<=EPS)return false;
12         r=(l.p*x-(l.p+l.pq)*y)/(x-y);
13         return true;
14     }
15     bool inter(plane w, ln& r){
16         pt nn=n^w.n;pt v=n^nn;double d=w.n*v;
17         if(abs(d)<=EPS)return false;
18         pt p=a+v*(w.n*(w.a-a)/d);
19         r=ln(p,p+nn);
20         return true;
21     }
22 };

```

5.6 Radial order of points

```

1 struct Cmp { // IMPORTANT: add const in pt operator -
2     pt r;
3     Cmp(pt r):r(r){}
4     int cuad(const pt &a)const {
5         if(a.x>0&&a.y>=0)return 0;
6         if(a.x<=0&&a.y>0)return 1;
7         if(a.x<0&&a.y<=0)return 2;
8         if(a.x>=0&&a.y<0)return 3;
9         assert(a.x==0&&a.y==0);
10        return -1;

```

```

11 }
12 bool cmp(const pt& p1, const pt& p2)const {
13     int c1=cuad(p1),c2=cuad(p2);
14     if(c1==c2)return p1.y*p2.x<p1.x*p2.y;
15     return c1<c2;
16 }
17 bool operator()(const pt& p1, const pt& p2)const {
18     return cmp(p1-r,p2-r);
19 }
20 };

```

5.7 Convex hull

```

1 // CCW order
2 // Includes collinear points (change sign of EPS in left to exclude)
3 vector<pt> hull(vector<pt> p){
4     vector<pt> r;
5     sort(p.begin(),p.end()); // first x, then y
6     p.erase(unique(ALL(p)), p.end());
7     for (int i = 0; i < p.size(); ++i){ // lower hull
8         while(r.size()>=2&&r.back().left(r[r.size()-2],p[i]))r.pop_back();
9         r.push_back(p[i]);
10    }
11    r.pop_back();
12    int k=r.size(),pr=k-1;
13    for(int i=p.size()-1;i>=0;--i){ // upper hull
14        while(r.size()>=k+2&&r.back().left(r[r.size()-2],p[i]))r.pop_back();
15        while(pr>=0&&p[i]<r[pr]) pr--;
16        if(pr<0||!(p[i]==r[pr])) r.push_back(p[i]);
17    }
18    return r;
19 }

```

5.8 Dual from planar graph

```

1 vector<int> g[MAXN];int n; // input graph (must be connected)
2 vector<int> gd[MAXN];int nd; // output graph
3 vector<int> nodes[MAXN]; // nodes delimiting region (in CW order)
4 map<pair<int,int>,int> ps,es;
5 void get_dual(vector<pt> p){ // p: points corresponding to nodes
6     ps.clear();es.clear();
7     for (int x = 0; x < n; ++x){
8         Cmp pc(p[x]); // (radial order of points)
9         auto comp=[&](int a, int b){return pc(p[a],p[b]);};

```

```

10     sort(g[x].begin(),g[x].end(),comp);
11     for (int i = 0; i < g[x].size(); ++i)ps[{x,g[x][i]}]=i;
12 }
13 nd=0;
14 for (int xx = 0; xx < n; ++xx)for(auto yy:g[xx])if(!es.count({xx,yy}))
15 {
16     int x=xx,y=yy;gd[nd].clear();nodes[nd].clear();
17     while(!es.count({x,y})){
18         es[{x,y}]=nd;nodes[nd].push_back(y);
19         int z=g[y][(ps[{y,x}]+1)%g[y].size()];x=y;y=z;
20     }
21     nd++;
22 }
23 for(auto p:es){
24     pair<int,int> q={p.first.second,p.first.first};
25     assert(es.count(q));
26     if(es[q]!=p.second)gd[p.second].push_back(es[q]);
27 }
28 for (int i = 0; i < nd; ++i){
29     sort(gd[i].begin(),gd[i].end());
30     gd[i].erase(unique(gd[i].begin(),gd[i].end()),gd[i].end());
31 }

```

5.9 Halfplane intersection

```

1 // polygon intersecting left side of halfplanes
2 struct halfplane:public ln{
3     double angle;
4     halfplane(){}
5     halfplane(pt a,pt b){p=a; pq=b-a; angle=atan2(pq.y,pq.x);}
6     bool operator<(halfplane b)const{return angle<b.angle;}
7     bool out(pt q){return pq%(q-p)<-EPS;}
8 };
9 vector<pt> intersect(vector<halfplane> b){
10     vector<pt>bx={{DINF,DINF},{-DINF,DINF},{-DINF,-DINF},{DINF,-DINF}};
11     for (int i = 0; i < 4; ++i) b.push_back(halfplane(bx[i],bx[(i+1)%4]));
12     sort(ALL(b));
13     int n=SZ(b),q=1,h=0;
14     vector<halfplane> c(SZ(b)+10);
15     for (int i = 0; i < n; ++i){
16         while(q<h&&b[i].out(c[h]^c[h-1])) h--;
17         while(q<h&&b[i].out(c[q]^c[q+1])) q++;

```

```

18     c[++h]=b[i];
19     if(q<h&&abs(c[h].pq*c[h-1].pq)<EPS){
20         if(c[h].pq*c[h-1].pq<=0) return {};
21         h--;
22         if(b[i].out(c[h].p)) c[h]=b[i];
23     }
24 }
25 while(q<h-1&&c[q].out(c[h]^c[h-1]))h--;
26 while(q<h-1&&c[h].out(c[q]^c[q+1]))q++;
27 if(h-q<=1)return {};
28 c[h+1]=c[q];
29 vector<pt> s;
30 for (int i = q; i < h+1; ++i) s.push_back(c[i]^c[i+1]);
31 return s;
32 }

```

5.10 KD tree

```

1 // given a set of points, answer queries of nearest point in O(log(n))
2 bool onx(pt a, pt b){return a.x<b.x;}
3 bool ony(pt a, pt b){return a.y<b.y;}
4 struct Node {
5     pt pp;
6     ll x0=INF, x1=-INF, y0=INF, y1=-INF;
7     Node *first=0, *second=0;
8     ll distance(pt p){
9         ll x=min(max(x0,p.x),x1);
10        ll y=min(max(y0,p.y),y1);
11        return (pt(x,y)-p).norm2();
12    }
13    Node(vector<pt>&& vp):pp(vp[0]){
14        for(pt p:vp){
15            x0=min(x0,p.x); x1=max(x1,p.x);
16            y0=min(y0,p.y); y1=max(y1,p.y);
17        }
18        if(SZ(vp)>1){
19            sort(ALL(vp),x1-x0>y1-y0?onx:ony);
20            int m=SZ(vp)/2;
21            first=new Node({vp.begin(),vp.begin()+m});
22            second=new Node({vp.begin()+m,vp.end()});
23        }
24    }
25 };

```

```

26 struct KDTree {
27     Node* root;
28     KDTree(const vector<pt>& vp):root(new Node({ALL(vp)})) {}
29     pair<ll,pt> search(pt p, Node *node){
30         if(!node->first){
31             //avoid query point as answer
32             //if(p==node->pp) {INF,pt()};
33             return {(p-node->pp).norm2(),node->pp};
34         }
35         Node *f=node->first, *s=node->second;
36         ll bf=f->distance(p), bs=s->distance(p);
37         if(bf>bs)swap(bf,bs),swap(f,s);
38         auto best=search(p,f);
39         if(bs<best.first) best=min(best,search(p,s));
40         return best;
41     }
42     pair<ll,pt> nearest(pt p){return search(p,root);}
43 };

```

5.11 Minkowski sum

```

1 // Compute Minkowski sum of two CONVEX polygons (remove collinear first)
2 // Returns answer in CCW order
3 void reorder(vector<pt> &p){
4     if(!p[2].left(p[0],p[1])) reverse(ALL(p));
5     int pos=0;
6     for (int i = 1; i < SZ(p); ++i) if(pt(p[i].y,p[i].x) < pt(p[pos].y,p[
7         pos].x)) pos=i;
8     rotate(p.begin(), p.begin()+pos, p.end());
9 }
10 vector<pt> minkowski_sum(vector<pt> p, vector<pt> q){
11     if(min(SZ(p),SZ(q))<3){
12         vector<pt> v;
13         for(pt pp:p) for(pt qq:q) v.push_back(pp+qq);
14         return hull(v);
15     }
16     reorder(p); reorder(q);
17     for (int i = 0; i < 2; ++i) p.push_back(p[i]), q.push_back(q[i]);
18     vector<pt> r;
19     int i=0,j=0;
20     while(i+2<SZ(p)||j+2<SZ(q)){
21         r.push_back(p[i]+q[j]);
22         auto cross=(p[i+1]-p[i])%(q[j+1]-q[j]);

```

```

22     i+=cross>=-EPS;
23     j+=cross<=EPS;
24 }
25 return r;
26 }

```

5.12 Delaunay triangulation

```

1 // Returns planar graph representing Delaunay's triangulation.
2 // Edges for each vertex are in ccw order.
3 // It can work with doubles, but also integers (replace long double in
4     line 51)
5 typedef struct QuadEdge* Q;
6 struct QuadEdge {
7     int id,used;
8     pt o;
9     Q rot,nxt;
10     QuadEdge(int id=-1, pt o=pt(INF,INF)):id(id),used(0),o(o),rot(0),nxt
11         (0){}
12     Q rev(){return rot->rot;}
13     Q next(){return nxt;}
14     Q prev(){return rot->next()->rot;}
15     pt dest(){return rev()->o;}
16 };
17 Q edge(pt a, pt b, int ida, int idb){
18     Q e1=new QuadEdge(ida,a);
19     Q e2=new QuadEdge(idb,b);
20     Q e3=new QuadEdge;
21     Q e4=new QuadEdge;
22     tie(e1->rot,e2->rot,e3->rot,e4->rot)={e3,e4,e2,e1};
23     tie(e1->nxt,e2->nxt,e3->nxt,e4->nxt)={e1,e2,e4,e3};
24     return e1;
25 }
26 void splice(Q a, Q b){
27     swap(a->nxt->rot->nxt,b->nxt->rot->nxt);
28     swap(a->nxt,b->nxt);
29 }
30 void del_edge(Q& e, Q ne){
31     splice(e,e->prev()); splice(e->rev(),e->rev()->prev());
32     delete e->rev()->rot; delete e->rev();
33 }

```



```

34 delete e->rot; delete e;
35 e=ne;
36 }
37
38 Q conn(Q a, Q b){
39     Q e=edge(a->dest(),b->o,a->rev()->id,b->id);
40     splice(e,a->rev()->prev());
41     splice(e->rev(),b);
42     return e;
43 }
44
45 auto area(pt p, pt q, pt r){return (q-p)%(r-q);}
46
47 // is p in circunference formed by (a,b,c)?
48 bool in_c(pt a, pt b, pt c, pt p){
49     // Warning: this number is 0(max_coord^4).
50     // Consider using int128 or using an alternative method for this
51     // function
52     long double p2=p*p,A=a*a-p2,B=b*b-p2,C=c*c-p2;
53     return area(p,a,b)*C+area(p,b,c)*A+area(p,c,a)*B>EPS;
54 }
55
56 pair<Q,Q> build_tr(vector<pt>& p, int l, int r){
57     if(r-l+1<=3){
58         Q a=edge(p[l],p[l+1],l,l+1),b=edge(p[l+1],p[r],l+1,r);
59         if(r-l+1==2) return {a,a->rev()};
60         splice(a->rev(),b);
61         auto ar=area(p[l],p[l+1],p[r]);
62         Q c=abs(ar)>EPS?conn(b,a):0;
63         if(ar>=-EPS) return {a,b->rev()};
64         return {c->rev(),c};
65     }
66     int m=(l+r)/2;
67     auto [la,ra]=build_tr(p,l,m);
68     auto [lb,rb]=build_tr(p,m+1,r);
69     while(1){
70         if(ra->dest().left(lb->o,ra->o)) ra=ra->rev()->prev();
71         else if(lb->dest().left(lb->o,ra->o)) lb=lb->rev()->next();
72         else break;
73     }
74     Q b=conn(lb->rev(),ra);
75     auto valid=[&](Q e){return b->o.left(e->dest(),b->dest());};
76     if(ra->o==la->o) la=b->rev();

```

```

76     if(lb->o==rb->o) rb=b;
77     while(1){
78         Q L=b->rev()->next();
79         if(valid(L)) while(in_c(b->dest(),b->o,L->dest(),L->next()->dest()))
80             del_edge(L,L->next());
81         Q R=b->prev();
82         if(valid(R)) while(in_c(b->dest(),b->o,R->dest(),R->prev()->dest()))
83             del_edge(R,R->prev());
84         if(!valid(L)&&!valid(R)) break;
85         if(!valid(L)|| (valid(R)&&in_c(L->dest(),L->o,R->o,R->dest())) b=
86             conn(R,b->rev());
87         else b=conn(b->rev(),L->rev());
88     }
89     return {la,rb};
90 }
91
92 vector<vector<int>> delaunay(vector<pt> v){
93     int n=SZ(v); auto tmp=v;
94     vector<int> id(n); iota(ALL(id),0);
95     sort(ALL(id), [&](int l, int r){return v[l]<v[r];});
96     for (int i = 0; i < n; ++i) v[i]=tmp[id[i]];
97     assert(unique(ALL(v))==v.end());
98     vector<vector<int>> g(n);
99     int col=1;
100     for (int i = 2; i < n; ++i) col&=abs(area(v[i],v[i-1],v[i-2]))<=EPS;
101     if(col){
102         for (int i = 1; i < n; ++i) g[id[i-1]].push_back(id[i]),g[id[i]].
103             push_back(id[i-1]);
104     }
105     else{
106         Q e=build_tr(v,0,n-1).first;
107         vector<Q> edg={e};
108         for(int i=0;i<SZ(edg);e=edg[i++]){
109             for(Q at=e;!at->used;at=at->next()){
110                 at->used=1;
111                 g[id[at->id]].push_back(id[at->rev()->id]);
112                 edg.push_back(at->rev());
113             }
114         }
115     }
116     return g;
117 }

```

6 Strings

6.1 KMP

```

1 vector<int> kmppre(string& t){ // r[i]: longest border of t[0,i]
2     vector<int> r(t.size()+1);r[0]=-1;
3     int j=-1;
4     for (int i = 0; i < t.size(); ++i){
5         while(j>=0&&t[i]!=t[j])j=r[j];
6         r[i+1]=++j;
7     }
8     return r;
9 }
10 void kmp(string& s, string& t){ // find t in s
11     int j=0;vector<int> b=kmppre(t);
12     for (int i = 0; i < s.size(); ++i){
13         while(j>=0&&s[i]!=t[j])j=b[j];
14         if(++j==t.size())printf("Match at %d\n",i-j+1),j=b[j];
15     }
16 }

```

6.2 Z function

```

1 vector<int> z_function(string& s){
2     int l=0,r=0,n=s.size();
3     vector<int> z(s.size(),0); // z[i] = max k: s[0,k] == s[i,i+k]
4     for (int i = 1; i < n; ++i){
5         if(i<=r)z[i]=min(r-i+1,z[i-l]);
6         while(i+z[i]<n&&s[z[i]]==s[i+z[i]])z[i]++;
7         if(i+z[i]-1>r)l=i,r=i+z[i]-1;
8     }
9     return z;
10 }

```

6.3 Manacher

```

1 int d1[MAXN]; //d1[i] = max odd palindrome centered on i
2 int d2[MAXN]; //d2[i] = max even palindrome centered on i
3 //s  aabbaacaabbaa
4 //d1 1111117111111
5 //d2 0103010010301
6 void manacher(string& s){
7     int l=0,r=-1,n=s.size();
8     for (int i = 0; i < n; ++i){

```

```

9         int k=i>r?1:min(d1[l+r-i],r-i);
10        while(i+k<n&&i-k>=0&&s[i+k]==s[i-k])k++;
11        d1[i]=k--;
12        if(i+k>r)l=i-k,r=i+k;
13    }
14    l=0;r=-1;
15    for (int i = 0; i < n; ++i){
16        int k=i>r?0:min(d2[l+r-i+1],r-i+1);k++;
17        while(i+k<n&&i-k>=0&&s[i+k-1]==s[i-k-1])k++;
18        d2[i]=--k;
19        if(i+k-1>r)l=i-k,r=i+k-1;
20    }
21 }

```

6.4 Aho-Corasick

```

1 struct vertex {
2     map<char,int> next,go;
3     int p,link;
4     char pch;
5     vector<int> leaf;
6     vertex(int p=-1, char pch=-1):p(p),pch(pch),link(-1){}
7 };
8 vector<vertex> t;
9 void aho_init(){ //do not forget!!
10     t.clear();t.push_back(vertex());
11 }
12 void add_string(string s, int id){
13     int v=0;
14     for(char c:s){
15         if(!t[v].next.count(c)){
16             t[v].next[c]=t.size();
17             t.push_back(vertex(v,c));
18         }
19         v=t[v].next[c];
20     }
21     t[v].leaf.push_back(id);
22 }
23 int go(int v, char c);
24 int get_link(int v){
25     if(t[v].link<0)
26         if(!v||!t[v].p)t[v].link=0;
27         else t[v].link=go(get_link(t[v].p),t[v].pch);

```

```

28     return t[v].link;
29 }
30 int go(int v, char c){
31     if(!t[v].go.count(c))
32         if(t[v].next.count(c))t[v].go[c]=t[v].next[c];
33     else t[v].go[c]=v==0?0:go(get_link(v),c);
34     return t[v].go[c];
35 }

```

6.5 Suffix automaton

```

1 struct state {int len,link;map<char,int> next;}; //clear next!!
2 state st[100005];
3 int sz,last;
4 void sa_init(){
5     last=st[0].len=0;sz=1;
6     st[0].link=-1;
7 }
8 void sa_extend(char c){
9     int k=sz++,p;
10    st[k].len=st[last].len+1;
11    for(p=last;p!=-1&&!st[p].next.count(c);p=st[p].link)st[p].next[c]=k;
12    if(p==-1)st[k].link=0;
13    else {
14        int q=st[p].next[c];
15        if(st[p].len+1==st[q].len)st[k].link=q;
16        else {
17            int w=sz++;
18            st[w].len=st[p].len+1;
19            st[w].next=st[q].next;st[w].link=st[q].link;
20            for(;p!=-1&&st[p].next[c]==q;p=st[p].link)st[p].next[c]=w;
21            st[q].link=st[k].link=w;
22        }
23    }
24    last=k;
25 }
26 // input: abcbcbc
27 // i,link,len,next
28 // 0 -1 0 (a,1) (b,5) (c,7)
29 // 1 0 1 (b,2)
30 // 2 5 2 (c,3)
31 // 3 7 3 (b,4)
32 // 4 9 4 (c,6)

```

```

33 // 5 0 1 (c,7)
34 // 6 11 5 (b,8)
35 // 7 0 2 (b,9)
36 // 8 9 6 (c,10)
37 // 9 5 3 (c,11)
38 // 10 11 7
39 // 11 7 4 (b,8)

```

6.6 Palindromic Tree

```

1 struct palindromic_tree{
2     static const int SIGMA=26;
3     struct Node{
4         int len, link, to[SIGMA];
5         ll cnt;
6         Node(int len, int link=0, ll cnt=1):len(len),link(link),cnt(cnt)
7         {
8             memset(to,0,sizeof(to));
9         }
10    };
11    vector<Node> ns;
12    int last;
13    palindromic_tree():last(0){ns.push_back(Node(-1));ns.push_back(Node(0));}
14    void add(int i, string &s){
15        int p=last, c=s[i]-'a';
16        while(s[i-ns[p].len-1]!=s[i])p=ns[p].link;
17        if(ns[p].to[c]){
18            last=ns[p].to[c];
19            ns[last].cnt++;
20        }else{
21            int q=ns[p].link;
22            while(s[i-ns[q].len-1]!=s[i])q=ns[q].link;
23            q=max(1,ns[q].to[c]);
24            last=ns[p].to[c]=SZ(ns);
25            ns.push_back(Node(ns[p].len+2,q,1));
26        }
27    };

```

6.7 Suffix array (shorter but slower)

```

1 pair<int, int> sf[MAXN];
2 bool sacomp(int lhs, int rhs) {return sf[lhs]<sf[rhs];}

```

```

3 vector<int> constructSA(string& s){ //  $O(n \log^2(n))$ 
4   int n=s.size(); // (sometimes fast enough)
5   vector<int> sa(n),r(n);
6   for (int i = 0; i < n; ++i)r[i]=s[i];
7   for(int m=1;m<n;m*=2){
8     for (int i = 0; i < n; ++i)sa[i]=i,sf[i]={r[i],i+m<n?r[i+m]:-1};
9     stable_sort(sa.begin(),sa.end(),sacomp);
10    r[sa[0]]=0;
11    for (int i = 1; i < n; ++i)r[sa[i]]=sf[sa[i]]!=sf[sa[i-1]]?i:r[sa[i-1]];
12  }
13  return sa;
14 }

```

6.8 Suffix array

```

1 #define RB(x) (x<n?r[x]:0)
2 void csort(vector<int>& sa, vector<int>& r, int k){
3   int n=sa.size(),top=max(255,n+1);
4   vector<int> f(top,0),t(n);
5   for (int i = 0; i < n; ++i)f[RB(i+k)]++;
6   int sum=0;
7   for (int i = 0; i < top; ++i)f[i]=(sum+=f[i])-f[i];
8   for (int i = 0; i < n; ++i)t[f[RB(sa[i]+k)]]+=sa[i];
9   sa=t;
10 }
11 vector<int> constructSA(string& s){ //  $O(n \log n)$ 
12   int n=s.size(),rank;
13   vector<int> sa(n),r(n),t(n);
14   for (int i = 0; i < n; ++i)sa[i]=i,r[i]=s[i];
15   for(int k=1;k<n;k*=2){
16     csort(sa,r,k);csort(sa,r,0);
17     t[sa[0]]=rank=1;
18     for (int i = 1; i < n; ++i){
19       if(r[sa[i]]!=r[sa[i-1]]||RB(sa[i]+k)!=RB(sa[i-1]+k))rank++;
20       t[sa[i]]=rank;
21     }
22     r=t;
23     if(r[sa[n-1]]==n)break;
24   }
25   return sa;
26 }

```

6.9 LCP (Longest Common Prefix)

```

1 vector<int> computeLCP(string& s, vector<int>& sa){
2   int n=s.size(),L=0;
3   vector<int> lcp(n),plcp(n),phi(n);
4   phi[sa[0]]=-1;
5   for (int i = 1; i < n; ++i)phi[sa[i]]=sa[i-1];
6   for (int i = 0; i < n; ++i){
7     if(phi[i]<0){plcp[i]=0;continue;}
8     while(s[i+L]==s[phi[i]+L])L++;
9     plcp[i]=L;
10    L=max(L-1,0);
11  }
12  for (int i = 0; i < n; ++i)lcp[i]=plcp[sa[i]];
13  return lcp; // lcp[i]=LCP(sa[i-1],sa[i])
14 }

```

6.10 Suffix Tree (Ukkonen's algorithm)

```

1 struct SuffixTree {
2   char s[MAXN];
3   map<int,int> to[MAXN];
4   int len[MAXN]={INF},fpos[MAXN],link[MAXN];
5   int node,pos,sz=1,n=0;
6   int make_node(int p, int l){
7     fpos[sz]=p;len[sz]=l;return sz++;}
8   void go_edge(){
9     while(pos>len[to[node][s[n-pos]]]){
10       node=to[node][s[n-pos]];
11       pos-=len[node];
12     }
13   }
14   void add(int c){
15     s[n++]=c;pos++;
16     int last=0;
17     while(pos>0){
18       go_edge();
19       int edge=s[n-pos];
20       int& v=to[node][edge];
21       int t=s[fpos[v]+pos-1];
22       if(v==0){
23         v=make_node(n-pos,INF);
24         link[last]=node;last=0;

```

```

25     }
26     else if(t==c){link[last]=node;return;}
27     else {
28         int u=make_node(fpos[v],pos-1);
29         to[u][c]=make_node(n-1,INF);
30         to[u][t]=v;
31         fpos[v]+=pos-1;len[v]-=pos-1;
32         v=u;link[last]=u;last=u;
33     }
34     if(node==0)pos--;
35     else node=link[node];
36 }
37 }
38 };

```

6.11 Hashing

```

1 struct Hash {
2     int P=1777771,MOD[2],PI[2];
3     vector<int> h[2],pi[2];
4     Hash(string& s){
5         MOD[0]=999727999;MOD[1]=1070777777;
6         PI[0]=325255434;PI[1]=10018302;
7         for (int k = 0; k < 2; ++k)h[k].resize(s.size()+1),pi[k].resize(s.
            size()+1);
8         for (int k = 0; k < 2; ++k){
9             h[k][0]=0;pi[k][0]=1;
10            ll p=1;
11            for (int i = 1; i < s.size()+1; ++i){
12                h[k][i]=(h[k][i-1]+p*s[i-1])%MOD[k];
13                pi[k][i]=(1LL*pi[k][i-1]*PI[k])%MOD[k];
14                p=(p*P)%MOD[k];
15            }
16        }
17    }
18    ll get(int s, int e){
19        ll h0=(h[0][e]-h[0][s]+MOD[0])%MOD[0];
20        h0=(1LL*h0*pi[0][s])%MOD[0];
21        ll h1=(h[1][e]-h[1][s]+MOD[1])%MOD[1];
22        h1=(1LL*h1*pi[1][s])%MOD[1];
23        return (h0<<32)|h1;
24    }
25 };

```

6.12 Hashing with ll (using __int128)

```

1 #define bint __int128
2 struct Hash {
3     bint MOD=212345678987654321LL,P=1777771,PI=106955741089659571LL;
4     vector<bint> h,pi;
5     Hash(string& s){
6         assert((P*PI)%MOD==1);
7         h.resize(s.size()+1);pi.resize(s.size()+1);
8         h[0]=0;pi[0]=1;
9         bint p=1;
10        for (int i = 1; i < s.size()+1; ++i){
11            h[i]=(h[i-1]+p*s[i-1])%MOD;
12            pi[i]=(pi[i-1]*PI)%MOD;
13            p=(p*P)%MOD;
14        }
15    }
16    ll get(int s, int e){
17        return (((h[e]-h[s]+MOD)%MOD)*pi[s])%MOD;
18    }
19 };

```

7 Flow

7.1 Matching (slower)

```

1 vector<int> g[MAXN]; // [0,n)->[0,m]
2 int n,m;
3 int mat[MAXM];bool vis[MAXN];
4 int match(int x){
5     if(vis[x])return 0;
6     vis[x]=true;
7     for(int y:g[x])if(mat[y]<0||match(mat[y])){mat[y]=x;return 1;}
8     return 0;
9 }
10 vector<pair<int,int> > max_matching(){
11     vector<pair<int,int> > r;
12     memset(mat,-1,sizeof(mat));
13     for (int i = 0; i < n; ++i)memset(vis,false,sizeof(vis)),match(i);
14     for (int i = 0; i < m; ++i)if(mat[i]>=0)r.push_back({mat[i],i});
15     return r;
16 }

```

7.2 Matching (Hopcroft-Karp)

```

1 vector<int> g[MAXN]; // [0,n)->[0,m)
2 int n,m;
3 int mt[MAXN],mt2[MAXN],ds[MAXN];
4 bool bfs(){
5     queue<int> q;
6     memset(ds,-1,sizeof(ds));
7     for (int i = 0; i < n; ++i)if(mt2[i]<0)ds[i]=0,q.push(i);
8     bool r=false;
9     while(!q.empty()){
10         int x=q.front();q.pop();
11         for(int y:g[x]){
12             if(mt[y]>=0&&ds[mt[y]]<0)ds[mt[y]]=ds[x]+1,q.push(mt[y]);
13             else if(mt[y]<0)r=true;
14         }
15     }
16     return r;
17 }
18 bool dfs(int x){
19     for(int y:g[x])if(mt[y]<0||ds[mt[y]]==ds[x]+1&&dfs(mt[y])){
20         mt[y]=x;mt2[x]=y;
21         return true;
22     }
23     ds[x]=1<<30;
24     return false;
25 }
26 int mm(){
27     int r=0;
28     memset(mt,-1,sizeof(mt));memset(mt2,-1,sizeof(mt2));
29     while(bfs()){
30         for (int i = 0; i < n; ++i)if(mt2[i]<0)r+=dfs(i);
31     }
32     return r;
33 }

```

7.3 Hungarian

```

1 typedef long double td; typedef vector<int> vi; typedef vector<td> vd;
2 const td INF=1e100;//for maximum set INF to 0, and negate costs
3 bool zero(td x){return fabs(x)<1e-9;}//change to x==0, for ints/ll
4 struct Hungarian{
5     int n; vector<vd> cs; vi L, R;

```

```

6     Hungarian(int N, int M):n(max(N,M)),cs(n,vd(n)),L(n),R(n){
7         for (int x = 0; x < N; ++x)for (int y = 0; y < M; ++y)cs[x][y]=
            INF;
8     }
9     void set(int x,int y,td c){cs[x][y]=c;}
10 td assign() {
11     int mat = 0; vd ds(n), u(n), v(n); vi dad(n), sn(n);
12     for (int i = 0; i < n; ++i)u[i]=*min_element(ALL(cs[i]));
13     for (int j = 0; j < n; ++j){v[j]=cs[0][j]-u[0];for (int i = 1; i < n
        ; ++i)v[j]=min(v[j],cs[i][j]-u[i]);}
14     L=R=vi(n, -1);
15     for (int i = 0; i < n; ++i)for (int j = 0; j < n; ++j)
16         if(R[j]==-1&&zero(cs[i][j]-u[i]-v[j])){L[i]=j;R[j]=i;mat++;break;}
17     for(;mat<n;mat++){
18         int s=0, j=0, i;
19         while(L[s] != -1)s++;
20         fill(ALL(dad),-1);fill(ALL(sn),0);
21         for (int k = 0; k < n; ++k)ds[k]=cs[s][k]-u[s]-v[k];
22         for(;;){
23             j = -1;
24             for (int k = 0; k < n; ++k)if(!sn[k]&&(j==-1||ds[k]<ds[j]))j
                =k;
25             sn[j] = 1; i = R[j];
26             if(i == -1) break;
27             for (int k = 0; k < n; ++k)if(!sn[k]){
28                 auto new_ds=ds[j]+cs[i][k]-u[i]-v[k];
29                 if(ds[k] > new_ds){ds[k]=new_ds;dad[k]=j;}
30             }
31         }
32         for (int k = 0; k < n; ++k)if(k!=j&&sn[k]){auto w=ds[k]-ds[j];v[
            k]+=w,u[R[k]]-=w;}
33         u[s] += ds[j];
34         while(dad[j]>=0){int d = dad[j];R[j]=R[d];L[R[j]]=j;j=d;}
35         R[j]=s;L[s]=j;
36     }
37     td value=0;for (int i = 0; i < n; ++i)value+=cs[i][L[i]];
38     return value;
39 }
40 };

```

7.4 Dinic

```

1 // Min cut: nodes with dist>=0 vs nodes with dist<0

```

```

2 // Matching MVC: left nodes with dist<0 + right nodes with dist>0
3 struct Dinic{
4     int nodes,src,dst;
5     vector<int> dist,q,work;
6     struct edge {int to,rev;ll f,cap;};
7     vector<vector<edge>> g;
8     Dinic(int x):nodes(x),g(x),dist(x),q(x),work(x){}
9     void add_edge(int s, int t, ll cap){
10         g[s].push_back((edge){t,SZ(g[t]),0,cap});
11         g[t].push_back((edge){s,SZ(g[s])-1,0,0});
12     }
13     bool dinic_bfs(){
14         fill(ALL(dist),-1);dist[src]=0;
15         int qt=0;q[qt++]=src;
16         for(int qh=0;qh<qt;qh++){
17             int u=q[qh];
18             for (int i = 0; i < SZ(g[u]); ++i){
19                 edge &e=g[u][i];int v=g[u][i].to;
20                 if(dist[v]<0&&e.f<e.cap)dist[v]=dist[u]+1,q[qt++]=v;
21             }
22         }
23         return dist[dst]>=0;
24     }
25     ll dinic_dfs(int u, ll f){
26         if(u==dst)return f;
27         for(int &i=work[u];i<SZ(g[u]);i++){
28             edge &e=g[u][i];
29             if(e.cap<=e.f)continue;
30             int v=e.to;
31             if(dist[v]==dist[u]+1){
32                 ll df=dinic_dfs(v,min(f,e.cap-e.f));
33                 if(df>0){e.f+=df;g[v][e.rev].f-=df;return df;}
34             }
35         }
36         return 0;
37     }
38     ll max_flow(int _src, int _dst){
39         src=_src;dst=_dst;
40         ll result=0;
41         while(dinic_bfs()){
42             fill(ALL(work),0);
43             while(ll delta=dinic_dfs(src,INF))result+=delta;
44         }

```

```

45     return result;
46 }
47 };

```

7.5 Min cost max flow

```

1 typedef ll tf;
2 typedef ll tc;
3 const tf INFFLOW=1e9;
4 const tc INFCOST=1e9;
5 struct MCF{
6     int n;
7     vector<tc> prio, pot; vector<tf> curflow; vector<int> prevedge,
8         prevnode;
9     priority_queue<pair<tc, int>, vector<pair<tc, int>>, greater<pair<tc,
10         int>>> q;
11     struct edge{int to, rev; tf f, cap; tc cost;};
12     vector<vector<edge>> g;
13     MCF(int n):n(n),prio(n),curflow(n),prevedge(n),prevnode(n),pot(n),g(n)
14         {}
15     void add_edge(int s, int t, tf cap, tc cost) {
16         g[s].push_back((edge){t,SZ(g[t]),0,cap,cost});
17         g[t].push_back((edge){s,SZ(g[s])-1,0,0,-cost});
18     }
19     pair<tf,tc> get_flow(int s, int t) {
20         tf flow=0; tc flowcost=0;
21         while(1){
22             q.push({0, s});
23             fill(ALL(prio),INFCOST);
24             prio[s]=0; curflow[s]=INFFLOW;
25             while(!q.empty()) {
26                 auto cur=q.top();
27                 tc d=cur.first;
28                 int u=cur.second;
29                 q.pop();
30                 if(d!=prio[u]) continue;
31                 for(int i=0; i<SZ(g[u]); ++i) {
32                     edge &e=g[u][i];
33                     int v=e.to;
34                     if(e.cap<=e.f) continue;
35                     tc nprio=prio[u]+e.cost+pot[u]-pot[v];
36                     if(prio[v]>nprio) {
37                         prio[v]=nprio;

```



```

35     q.push({nprio, v});
36     prevnode[v]=u; prevedge[v]=i;
37     curflow[v]=min(curflow[u], e.cap-e.f);
38 }
39 }
40 }
41 if(prio[t]==INFCOST) break;
42 for (int i = 0; i < n; ++i) pot[i]+=prio[i];
43 tf df=min(curflow[t], INFFLOW-flow);
44 flow+=df;
45 for(int v=t; v!=s; v=prevnode[v]) {
46     edge &e=g[prevnode[v]][prevedge[v]];
47     e.f+=df; g[v][e.rev].f-=df;
48     flowcost+=df*e.cost;
49 }
50 }
51 return {flow,flowcost};
52 }
53 };

```

8 Other

8.1 Mo's algorithm

```

1 int n,sq,nq; // array size, sqrt(array size), #queries
2 struct qu{int l,r,id;};
3 qu qs[MAXN];
4 ll ans[MAXN]; // ans[i] = answer to ith query
5 bool qcomp(const qu &a, const qu &b){
6     if(a.l/sq!=b.l/sq) return a.l<b.l;
7     return (a.l/sq)&1?a.r<b.r:a.r>b.r;
8 }
9 void mos(){
10     for (int i = 0; i < nq; ++i)qs[i].id=i;
11     sq=sqrt(n)+.5;
12     sort(qs,qs+nq,qcomp);
13     int l=0,r=0;
14     init();
15     for (int i = 0; i < nq; ++i){
16         qu q=qs[i];
17         while(l>q.l)add(--l);
18         while(r<q.r)add(r++);
19         while(l<q.l)remove(l++);

```

```

20         while(r>q.r)remove(--r);
21         ans[q.id]=get_ans();
22     }
23 }

```

8.2 Divide and conquer DP optimization

```

1 // O(knlogn). For 2D dps, when the position of optimal choice is non-
  // decreasing as the second variable increases
2 int k,n,f[MAXN],f2[MAXN];
3 void doit(int s, int e, int s0, int e0, int i){
4     // [s,e): range of calculation, [s0,e0): range of optimal choice
5     if(s==e)return;
6     int m=(s+e)/2,r=INF,rp;
7     for (int j = s0; j < min(e0,m); ++j){
8         int r0=something(i,j); // "something" usually depends on f
9         if(r0<r)r=r0,rp=j; // position of optimal choice
10    }
11    f2[m]=r;
12    doit(s,m,s0,rp+1,i);doit(m+1,e,rp,e0,i);
13 }
14 int doall(){
15     init_base_cases();
16     for (int i = 1; i < k; ++i)doit(1,n+1,0,n,i),memcpy(f,f2,sizeof(f));
17     return f[n];
18 }

```

8.3 Dates

```

1 int dateToInt(int y, int m, int d){
2     return 1461*(y+4800+(m-14)/12)/4+367*(m-2-(m-14)/12*12)/12-
3         3*((y+4900+(m-14)/12)/100)/4+d-32075;
4 }
5 void intToDate(int jd, int& y, int& m, int& d){
6     int x,n,i,j;x=jd+68569;
7     n=4*x/146097;x-=(146097*n+3)/4;
8     i=(4000*(x+1))/1461001;x-=1461*i/4-31;
9     j=80*x/2447;d=x-2447*j/80;
10    x=j/11;m=j+2-12*x;y=100*(n-49)+i+x;
11 }
12 int DayOfWeek(int d, int m, int y){ //starting on Sunday
13     static int ttt[]={0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4};
14     y-=m<3;
15     return (y+y/4-y/100+y/400+ttt[m-1]+d)%7;

```

16 } }

8.4 C++ stuff

```

1 // double inf
2 const double DINF=numeric_limits<double>::infinity();
3 // Custom comparator for set/map
4 struct comp {
5     bool operator()(const double& a, const double& b) const {
6         return a+EPS<b;}
7 };
8 set<double,comp> w; // or map<double,int,comp>
9 // Iterate over non empty subsets of bitmask
10 for(int s=m;s;s=(s-1)&m) // Decreasing order
11 for (int s=0;s=s-m&m;) // Increasing order
12 // Return the numbers the numbers of 1-bit in x
13 int __builtin_popcount (unsigned int x)
14 // Returns the number of trailing 0-bits in x. x=0 is undefined.
15 int __builtin_ctz (unsigned int x)
16 // Returns the number of leading 0-bits in x. x=0 is undefined.
17 int __builtin_clz (unsigned int x)
18 // x of type long long just add 'll' at the end of the function.
19 int __builtin_popcountll (unsigned long long x)
20 // Get the value of the least significant bit that is one.
21 v=(x&(-x))

```

8.5 Interactive problem tester template

```

1 #Easier method with bash commands:
2 #mkfifo fifo
3 #(. /solution < fifo) | (. /interactor > fifo)
4
5 # tester for cf 101021A (guess a number, queries: is it >=k?)
6 import random
7 import subprocess as sp
8 seed = random.randint(0, sys.maxint);random.seed(seed)
9 n=random.randint(1,1000000)
10 try:
11     p=sp.Popen(['./a.out'],stdin=sp.PIPE,stdout=sp.PIPE)
12     it=0
13     s=p.stdout.readline()
14     while it<25 and s and s[0]!='!':
15         k=int(s)
16         assert k>=1 and k<=1000000

```

```

17     if n>=k: p.stdin.write('>=\\n')
18     else: p.stdin.write('<\\n')
19     s=p.stdout.readline()
20     it+=1
21     assert s and s[0]=='!'
22     k=int(s.split()[1])
23     assert k==n
24 except:
25     print 'failed with seed %s' % seed
26     raise

```

8.6 Max number of divisors up to 10ⁿ

```

1 (0,1) (1,4) (2,12) (3,32) (4,64) (5,128) (6,240) (7,448) (8,768)
   (9,1344) (10,2304) (11,4032) (12,6720) (13,10752) (14,17280)
   (15,26880) (16,41472) (17,64512) (18,103680)

```

8.7 Template

```

1 #include <bits/stdc++.h>
2 #ifdef BRAULIO
3 #define deb(...) fprintf(stderr,__VA_ARGS__)
4 #define deb1(x) cerr << #x << " = " << x << endl
5 #else
6 #define deb(...) 0
7 #define deb1(x) 0
8 #endif
9
10 // --- DEBUGGING TOOLS ---
11 void void_debug() { std::cerr << std::endl; }
12 template<typename Head, typename... Tail>
13 void void_debug(const Head& H, const Tail&... T) { std::cerr << " " << H
   ; void_debug(T...); }
14
15 #ifdef LOCAL
16 #define debug(...) std::cerr << "[" << #__VA_ARGS__ << "]:", void_debug(
   __VA_ARGS__)
17 #else
18 #define debug(...) 42
19 #endif
20
21 /*
22 Uso en competencia:

```

```
23 | Compila localmente con el flag -DLOCAL para activar las salidas por cerr
    | , por
24 | ejemplo:
25 | g++ -std=c++17 -O2 -Wall -Wextra -DLOCAL main.cpp -o main
26 |
27 | Dentro del codigo puedes escribir:
28 |     debug(variable_x, mi_vector[0], "llegue aqui");
29 |
30 | Cuando LOCAL no esta definido, como ocurre normalmente al enviar al juez
    | en
31 | linea, debug(...) se expande a una expresion inocua y no imprime nada.
    | Esto
32 | mantiene las trazas fuera del output oficial sin tener que borrar
    | llamadas de
33 | debug antes de enviar gracias a #ifdef LOCAL. Si tu entorno usa
34 | #ifndef ONLINE_JUDGE, puedes mantener la misma idea: compila localmente
    | con
35 | -DLOCAL y envia sin ese flag.
36 | */
37 |
38 | #define SZ(x) ((int)x.size())
39 | using namespace std;
40 | typedef long long ll;
41 |
42 | int main(){
43 |     return 0;
44 | }
```